



面向物联网的开源RISC-V 处理器的设计和开发

芯来科技 胡振波

自我介绍

- 国内领先的RISC-V处理器内核公司芯来科技CEO兼创始人
- 国内第一颗RISC-V开源处理器蜂鸟E203作者；第一本RISC-V中文书籍《手把手教你设计CPU—RISC-V处理器篇》和第二本书《RISC-V架构与嵌入式开发快速入门》作者
- 业界超过8年CPU以及超过10年ASIC设计与验证经验。历任Marvell(美满电子科技)CPU高级设计工程师， Synopsys(新思科技)ARC系列处理器内核研发经理等职务
- 上海交通大学微电子学院硕士，电子工程系学士



Agenda

- 1 关于ISA和RISC-V
- 2 开源RISC-V内核蜂鸟E203
- 3 RISC-V的中文书籍
- 4 蜂鸟E203软硬件DEMO视频
- 5 RISC-V的几个误区
- 6 RISC-V的未来趋势
- 7 RISC-V的商业本质
- 8 芯来科技介绍

Agenda

- 1 关于ISA和RISC-V**
- 2 开源RISC-V内核蜂鸟E203
- 3 RISC-V的中文书籍
- 4 蜂鸟E203软硬件DEMO视频
- 5 RISC-V的几个误区
- 6 RISC-V的未来趋势
- 7 RISC-V的商业本质
- 8 芯来科技介绍

指令集架构

- 指令集架构（ISA, Instruction Set Architecture），或者称之为“处理器架构”，简称为“架构”是**处理器的灵魂**；
- 指令集架构不仅仅是一组指令的集合，它还要定义任何软件程序员需要了解的硬件信息，包括支持的数据类型、存储器（Memory）和寄存器状态、寻址模式与存储器模型等；
- 其可以理解为一个抽象层，构成处理器底层硬件与运行于其上的软件之间的桥梁与接口，也是现在计算机处理器中最重要的一个抽象层。

软件

ISA

硬件



指令集架构和微架构

基于指令集架构ISA实现的具体的处理器硬件实现叫**微架构(Microarchitecture)**，是需要和指令集架构区分的一个概念。假设我们定义一个叫“车”的指令集，能够实现启动，加速，换挡，刹车等操作，基于这个指令集设计的具体的车，也会根据性能，功能，体验，载客量等产生不同类型车，而每种类型的车其结构和动力千差万别。基于同一指令集，针对性能、功耗和面积不同要求的处理器，在取指、流水线长度、缓存等结构设计上有所差别。



架构乐园

- 指令集架构风格分复杂指令集CISC和精简指令集RISC。CISC包含大量不常用指令，而RISC只包含最常用指令。但典型程序里使用的80%的指令只占有所有指令类型的20%，这导致CISC处理器设计复杂，增大了成本和面积开销，让RISC成了现代处理器的主流。包括x86也通过微码化实现了内部的RISC化。
- 目前市面上能够看到的主流架构包括x86, ARM等，但都面临问题包括 **“不开放”**，多为某个公司拥有，另外就是有历史包袱，不够精简。那么能否有一套标准指令集能够被任何学术和商业组织自由使用的吗？

ARM

ARC

ANDES

ALPHA

C-SKY

MIPS

POWER

SPARC

x86



RISC-V的诞生

RISC-V是总结了之前指令集架构后，设计出了简洁模块化可扩展的新的指令集架构。在基础指令集基础上，用户可以根据自己的应用对指令集进行扩展或裁剪以面向市场的不同需求。

基础指令集	指令数	描述
RV32I	47	32位整数指令集
RV32E	47	32位整数指令集，限制寄存器数量为16
RV64I	59	64位整数指令集(附带32位整数指令集)
RV128I	71	128位整数指令集(附带64位和32位整数指令集)
扩展指令集	指令数	描述
M	8	整数乘除法指令集
A	11	原子操作指令集
F	26	32位单浮点运算指令集
D	26	64位双精度浮点运算指令集(附带F指令集)
C	46	16位压缩模式的整数指令集

1998, 1999, 2000, 2001, 2002, 2003, 2004, 2005, 2006, 2007, 2008, 2009, 2010, 2011, 2012, 2013, 2014, 2015, 2016, 2017, 2018, 2019, 2020, 2021, 2022, 2023, 2024, 2025, 2026, 2027, 2028, 2029, 2030, 2031, 2032, 2033, 2034, 2035, 2036, 2037, 2038, 2039, 2040, 2041, 2042, 2043, 2044, 2045, 2046, 2047, 2048, 2049, 2050, 2051, 2052, 2053, 2054, 2055, 2056, 2057, 2058, 2059, 2060, 2061, 2062, 2063, 2064, 2065, 2066, 2067, 2068, 2069, 2070, 2071, 2072, 2073, 2074, 2075, 2076, 2077, 2078, 2079, 2080, 2081, 2082, 2083, 2084, 2085, 2086, 2087, 2088, 2089, 2090, 2091, 2092, 2093, 2094, 2095, 2096, 2097, 2098, 2099, 2100, 2101, 2102, 2103, 2104, 2105, 2106, 2107, 2108, 2109, 2110, 2111, 2112, 2113, 2114, 2115, 2116, 2117, 2118, 2119, 2120, 2121, 2122, 2123, 2124, 2125, 2126, 2127, 2128, 2129, 2130, 2131, 2132, 2133, 2134, 2135, 2136, 2137, 2138, 2139, 2140, 2141, 2142, 2143, 2144, 2145, 2146, 2147, 2148, 2149, 2150, 2151, 2152, 2153, 2154, 2155, 2156, 2157, 2158, 2159, 2160, 2161, 2162, 2163, 2164, 2165, 2166, 2167, 2168, 2169, 2170, 2171, 2172, 2173, 2174, 2175, 2176, 2177, 2178, 2179, 2180, 2181, 2182, 2183, 2184, 2185, 2186, 2187, 2188, 2189, 2190, 2191, 2192, 2193, 2194, 2195, 2196, 2197, 2198, 2199, 2200, 2201, 2202, 2203, 2204, 2205, 2206, 2207, 2208, 2209, 2210, 2211, 2212, 2213, 2214, 2215, 2216, 2217, 2218, 2219, 2220, 2221, 2222, 2223, 2224, 2225, 2226, 2227, 2228, 2229, 2230, 2231, 2232, 2233, 2234, 2235, 2236, 2237, 2238, 2239, 2240, 2241, 2242, 2243, 2244, 2245, 2246, 2247, 2248, 2249, 2250, 2251, 2252, 2253, 2254, 2255, 2256, 2257, 2258, 2259, 2260, 2261, 2262, 2263, 2264, 2265, 2266, 2267, 2268, 2269, 2270, 2271, 2272, 2273, 2274, 2275, 2276, 2277, 2278, 2279, 2280, 2281, 2282, 2283, 2284, 2285, 2286, 2287, 2288, 2289, 2290, 2291, 2292, 2293, 2294, 2295, 2296, 2297, 2298, 2299, 2300, 2301, 2302, 2303, 2304, 2305, 2306, 2307, 2308, 2309, 2310, 2311, 2312, 2313, 2314, 2315, 2316, 2317, 2318, 2319, 2320, 2321, 2322, 2323, 2324, 2325, 2326, 2327, 2328, 2329, 2330, 2331, 2332, 2333, 2334, 2335, 2336, 2337, 2338, 2339, 2340, 2341, 2342, 2343, 2344, 2345, 2346, 2347, 2348, 2349, 2350, 2351, 2352, 2353, 2354, 2355, 2356, 2357, 2358, 2359, 2360, 2361, 2362, 2363, 2364, 2365, 2366, 2367, 2368, 2369, 2370, 2371, 2372, 2373, 2374, 2375, 2376, 2377, 2378, 2379, 2380, 2381, 2382, 2383, 2384, 2385, 2386, 2387, 2388, 2389, 2390, 2391, 2392, 2393, 2394, 2395, 2396, 2397, 2398, 2399, 2400, 2401, 2402, 2403, 2404, 2405, 2406, 2407, 2408, 2409, 2410, 2411, 2412, 2413, 2414, 2415, 2416, 2417, 2418, 2419, 2420, 2421, 2422, 2423, 2424, 2425, 2426, 2427, 2428, 2429, 2430, 2431, 2432, 2433, 2434, 2435, 2436, 2437, 2438, 2439, 2440, 2441, 2442, 2443, 2444, 2445, 2446, 2447, 2448, 2449, 2450, 2451, 2452, 2453, 2454, 2455, 2456, 2457, 2458, 2459, 2460, 2461, 2462, 2463, 2464, 2465, 2466, 2467, 2468, 2469, 2470, 2471, 2472, 2473, 2474, 2475, 2476, 2477, 2478, 2479, 2480, 2481, 2482, 2483, 2484, 2485, 2486, 2487, 2488, 2489, 2490, 2491, 2492, 2493, 2494, 2495, 2496, 2497, 2498, 2499, 2500, 2501, 2502, 2503, 2504, 2505, 2506, 2507, 2508, 2509, 2510, 2511, 2512, 2513, 2514, 2515, 2516, 2517, 2518, 2519, 2520, 2521, 2522, 2523, 2524, 2525, 2526, 2527, 2528, 2529, 2530, 2531, 2532, 2533, 2534, 2535, 2536, 2537, 2538, 2539, 2540, 2541, 2542, 2543, 2544, 2545, 2546, 2547, 2548, 2549, 2550, 2551, 2552, 2553, 2554, 2555, 2556, 2557, 2558, 2559, 2560, 2561, 2562, 2563, 2564, 2565, 2566, 2567, 2568, 2569, 2570, 2571, 2572, 2573, 2574, 2575, 2576, 2577, 2578, 2579, 2580, 2581, 2582, 2583, 2584, 2585, 2586, 2587, 2588, 2589, 2590, 2591, 2592, 2593, 2594, 2595, 2596, 2597, 2598, 2599, 2600, 2601, 2602, 2603, 2604, 2605, 2606, 2607, 2608, 2609, 2610, 2611, 2612, 2613, 2614, 2615, 2616, 2617, 2618, 2619, 2620, 2621, 2622, 2623, 2624, 2625, 2626, 2627, 2628, 2629, 2630, 2631, 2632, 2633, 2634, 2635, 2636, 2637, 2638, 2639, 2640, 2641, 2642, 2643, 2644, 2645, 2646, 2647, 2648, 2649, 2650, 2651, 2652, 2653, 2654, 2655, 2656, 2657, 2658, 2659, 2660, 2661, 2662, 2663, 2664, 2665, 2666, 2667, 2668, 2669, 2670, 2671, 2672, 2673, 2674, 2675, 2676, 2677, 2678, 2679, 26

至今有接近200家企业加入基金会

Agenda

- 1 关于ISA和RISC-V
- 2 开源RISC-V内核蜂鸟E203**
- 3 RISC-V的中文书籍
- 4 蜂鸟E203软硬件DEMO视频
- 5 RISC-V的几个误区
- 6 RISC-V的未来趋势
- 7 RISC-V的商业本质
- 8 芯来科技介绍

处理器设计和验证的特点

处理器设计和验证是一个软硬件联合的过程，需具备系统知识，涉及的方面多，工作量大



想成为处理器设计工程师？

- 熟悉汇编语言以及执行过程
- 了解软件如何编译，汇编，链接最后成为处理器可执行的二进制的过程
- 了解计算机体系结构相关知识
- 处理器对时序和面积的要求一般会非常严格，需要不断地优化时序和面积，因此对电路和逻辑设计的理解要比较深刻



想成为处理器验证工程师？

- 使用UVM等传统的模块级验证手段对处理器子模块进行验证
- 需使用人工编写或随机生成的汇编语言测试用例在处理器上运行进行验证
- 需使用高级语言如C，C++等编写测试用例在处理器上进行验证

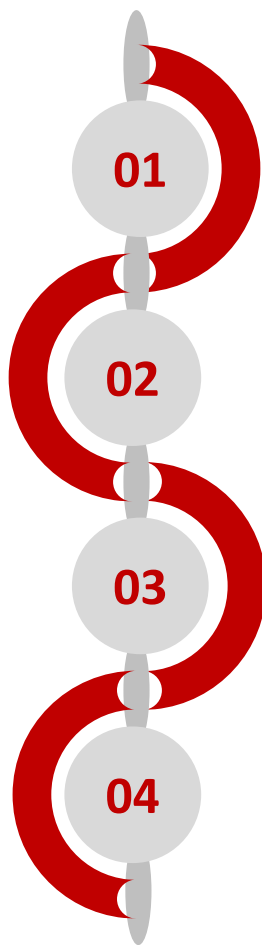
蜂鸟E203的设计目标和哲学

1.模块化和可重用性

将处理器划分为几个模块单元，每个单元接口简单清晰，并把复杂关系控制在单元内。划分的时候充分考虑可重用性，是其能够被其他微处理器内核架构继续使用，提升研发效率，降低成本。

3.结构简单化

为防止微架构设计陷入繁复的陷阱，在有选择的情况下优选最简单，关键场景才用复杂设计，遵循好钢用在刀刃上原则。



2.面积最小化

满足一定性能指标前提下，追求低功耗，小面积，因此设计中尽可能的复用数据通路节省面积开销。

4.性能不追求极端

定位为超低功耗处理器，以面积最小化和结构简单化为前提，性能提倡够用就行原则。

蜂鸟E203开源项目

- 纯Verilog HDL工业级标准开发，软硬件全套开源；
- 国内第一个完善的开源RISC-V项目，且众人关注；
- RISC-V基金会网站收录的国内唯一开源项目；
- 在国内的科研和教学领域广泛应用；
- 项目配套完整，除了代码和工具链还有完整的文档，书籍和开发板。

Documentation of Hummingbird E203

There will be three main docs to help user ramp up using the Hummingbird E203 Processor Core:

- 蜂鸟E203快速上手介绍.pdf
- 蜂鸟E203开源内核简介.pdf
- 蜂鸟E203开源SoC简介.pdf

Besides, there are two more docs under different directories which is very useful:

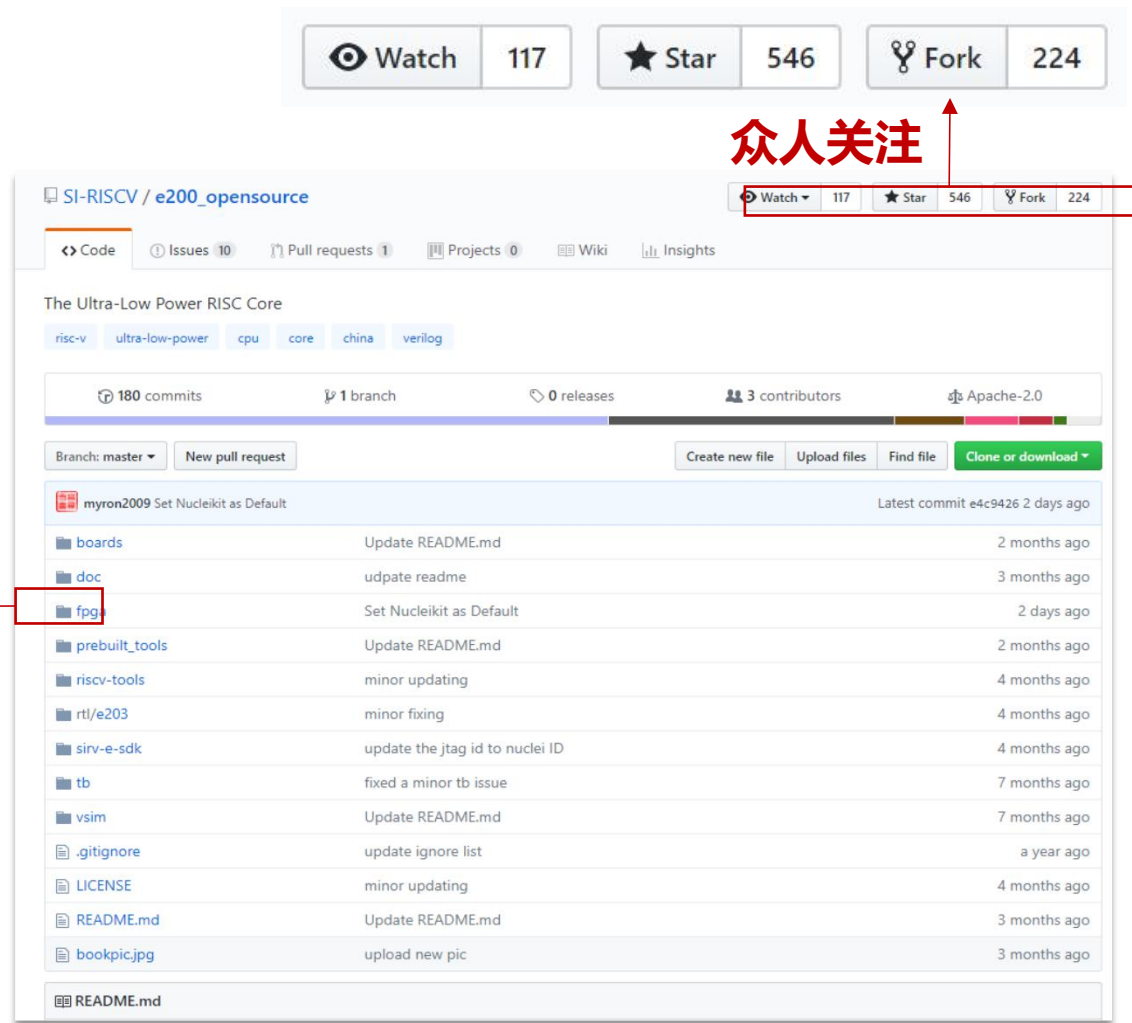
- https://github.com/SI-RISCV/e200_opensource/tree/master/boards/ 蜂鸟FPGA开发板和JTAG调试器介绍.pdf
- <https://github.com/SI-RISCV/hbird-e-sdk/tree/master/software/FreeRTOSv9.0.0/> 蜂鸟E203移植FreeRTOS.pdf

The Hummingbird E203 Core was deeply introduced in the published Book (蜂鸟E200处理器核在如下出版中文书籍中进行深入浅出的分析讲解):

《手把手教你设计CPU：RISC-V处理器》（已经上市，请在京东、淘宝、当当上搜索 RISC-V关键字）

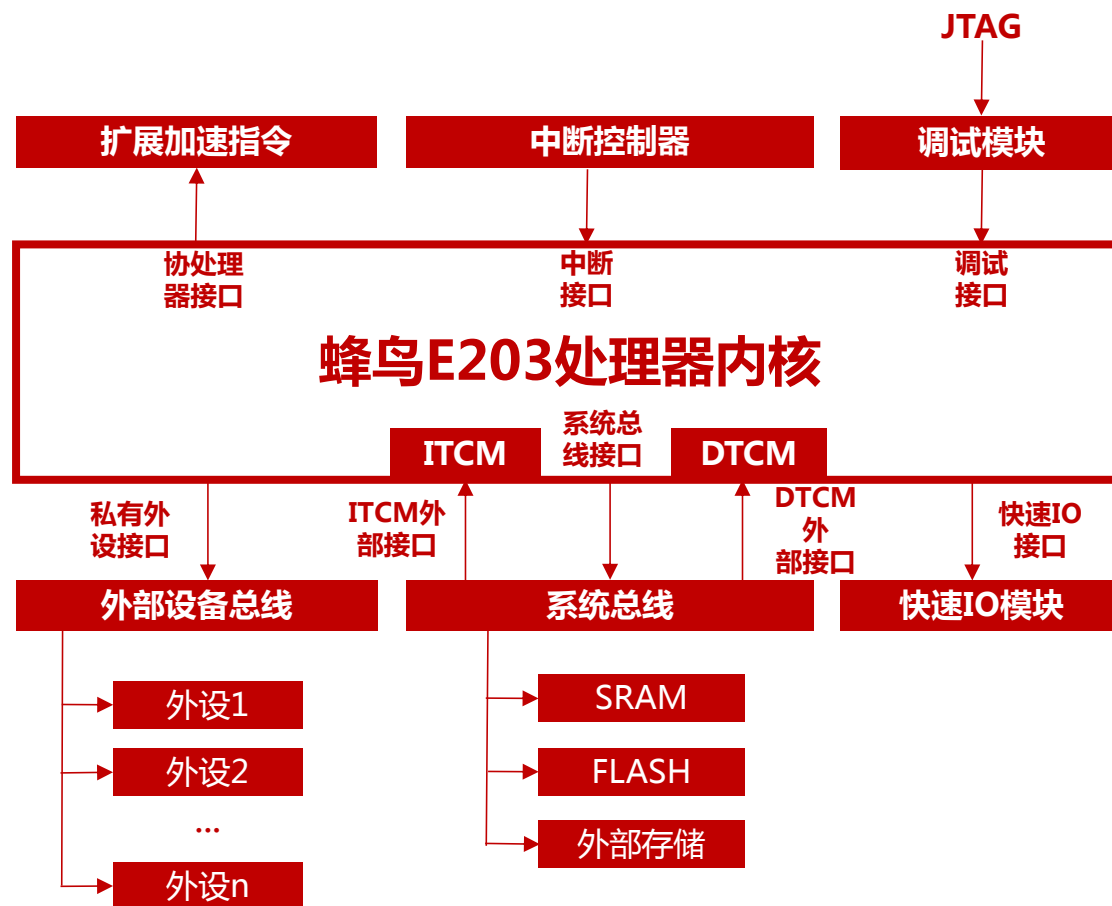
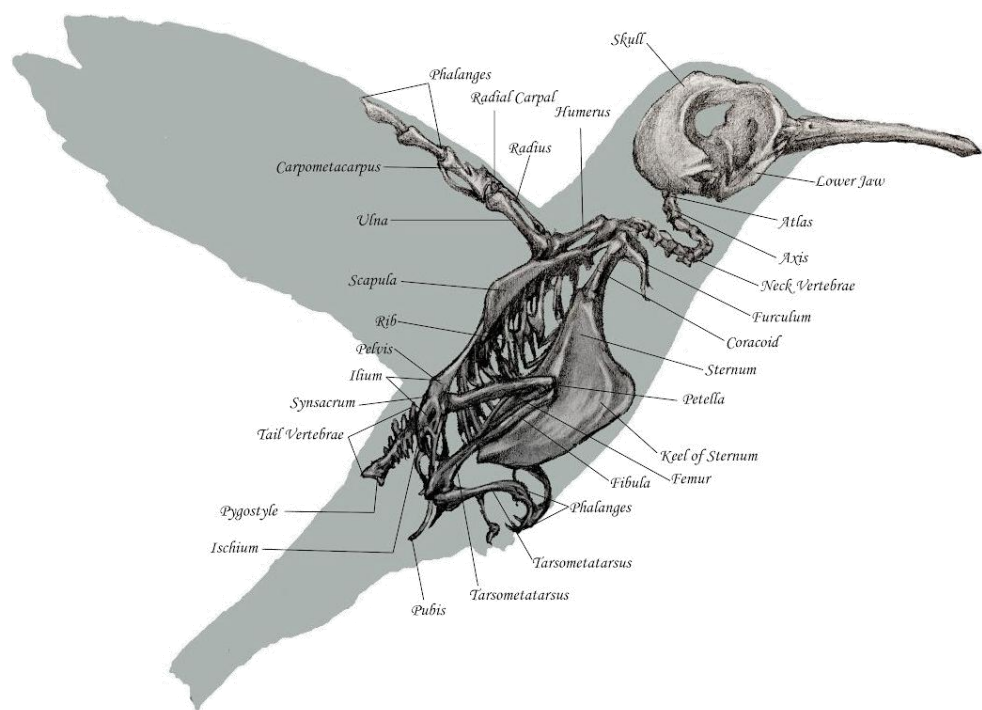


完善的文档



蜂鸟E203系统结构

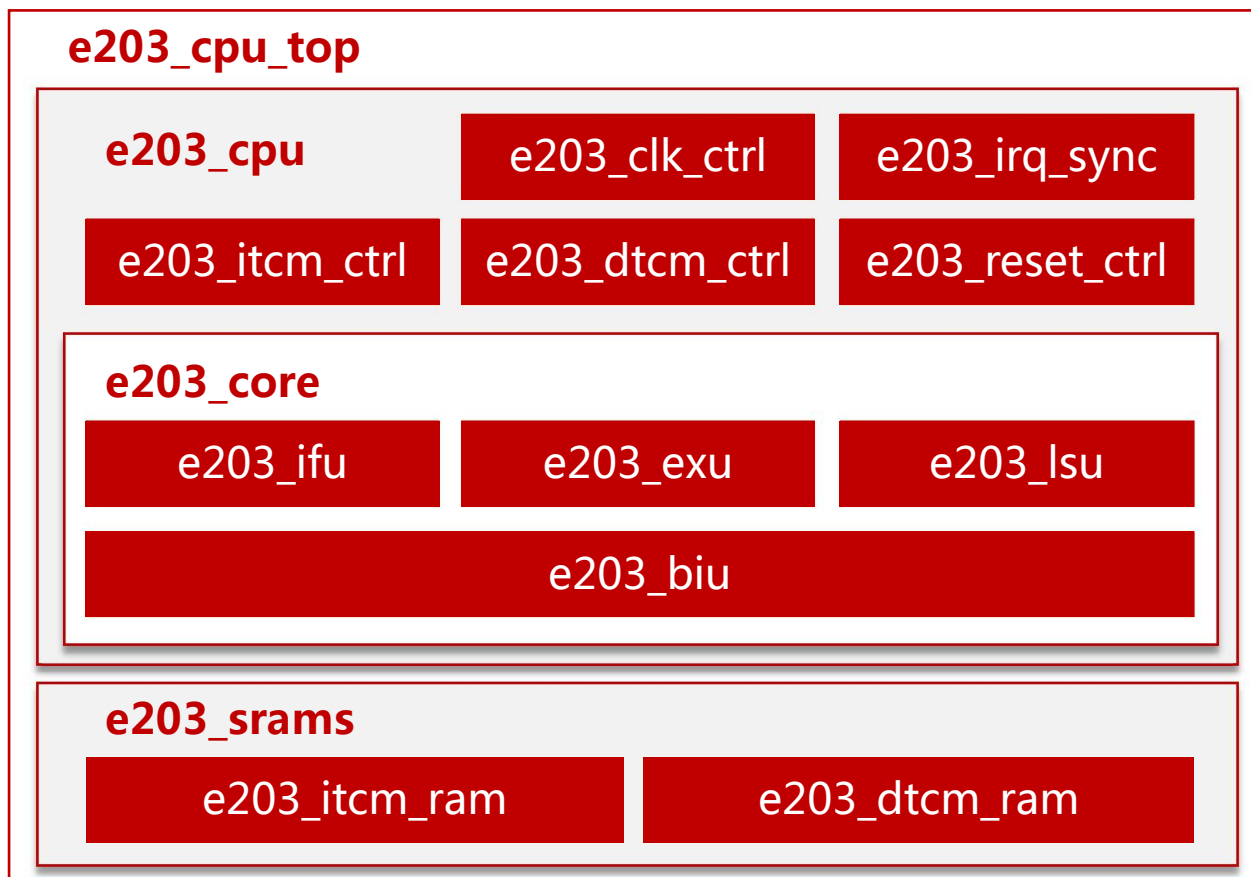
蜂鸟虽小，五脏俱全。蜂鸟E203内核虽精简，但源代码，工具链，文档和相关书籍一应俱全。该内核基于RV32IMAC架构设计，微架构为两级管线，单发射，指令和数据存储分离，支持机器模式，中断控制器接口，调试接口，指令扩展机制，以及配套的SoC。



蜂鸟E203模块层次划分

e203_cpu_top是蜂鸟E203顶层模块

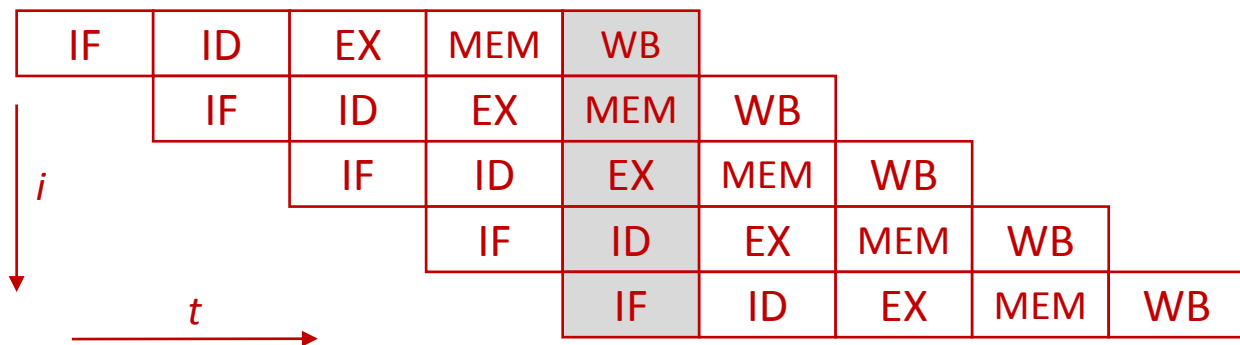
- **e203_cpu**，逻辑模块顶层
 - **e203_core**，处理器核
 - **e203_ifu**，取指单元IFU
 - **e203_exu**，执行单元EXU
 - **e203_lsu**，存储器访问单元LSU
 - **e203_biu**，总线接口单元BIU
 - **e203_clk_ctrl**，控制各组件自动时钟门控
 - **e203_irq_sync**，异步中断转同步
 - **e203_itcm_ctrl**，ITCM访问控制
 - **e203_dtcn_ctrl**，DTCM访问控制
 - **e203_reset_ctrl**，将外部异步置位信号同步
- **e203_srams**，SRAM模块顶层
 - **e203_itcm_ram**，ITCM指令紧耦合存储器
 - **e203_dtcn_ram**，DTCM数据紧耦合存储器



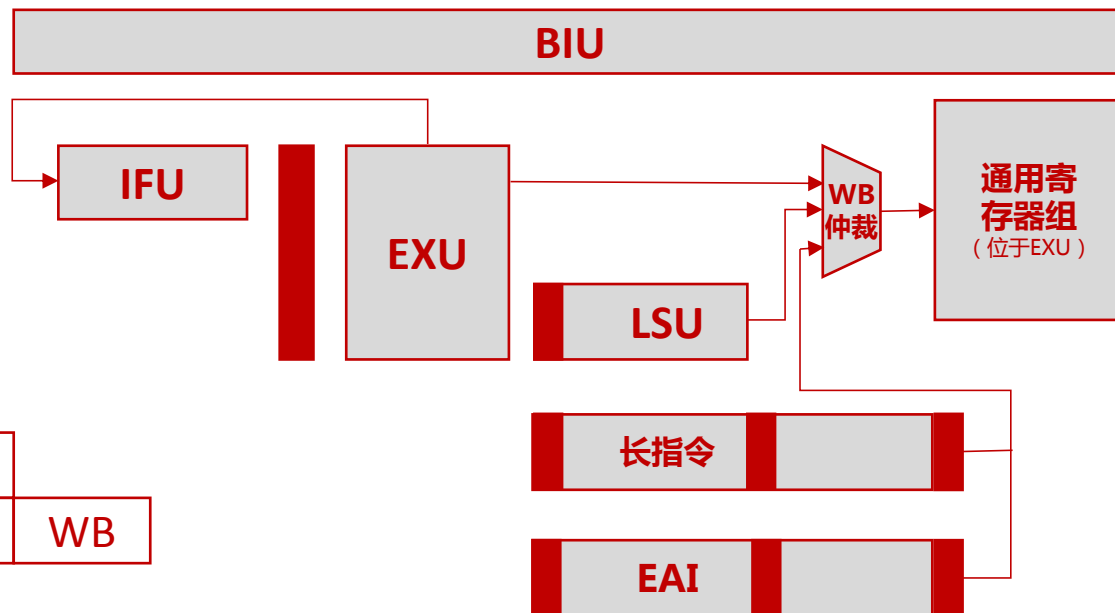
蜂鸟E203流水线设计

经典的五级流水线设计,一条指令生命周期分为5个步骤

- 取指(IF)**：从存储器中取出指令
- 译码(ID)**：对指令进行解析获取操作类型和操作数
- 执行(EX)**：对译码后的指令进行运算
- 访存(MEM)**：从存储器中读出或写入数据
- 写回(WB)**：将执行结果写回通用寄存器组



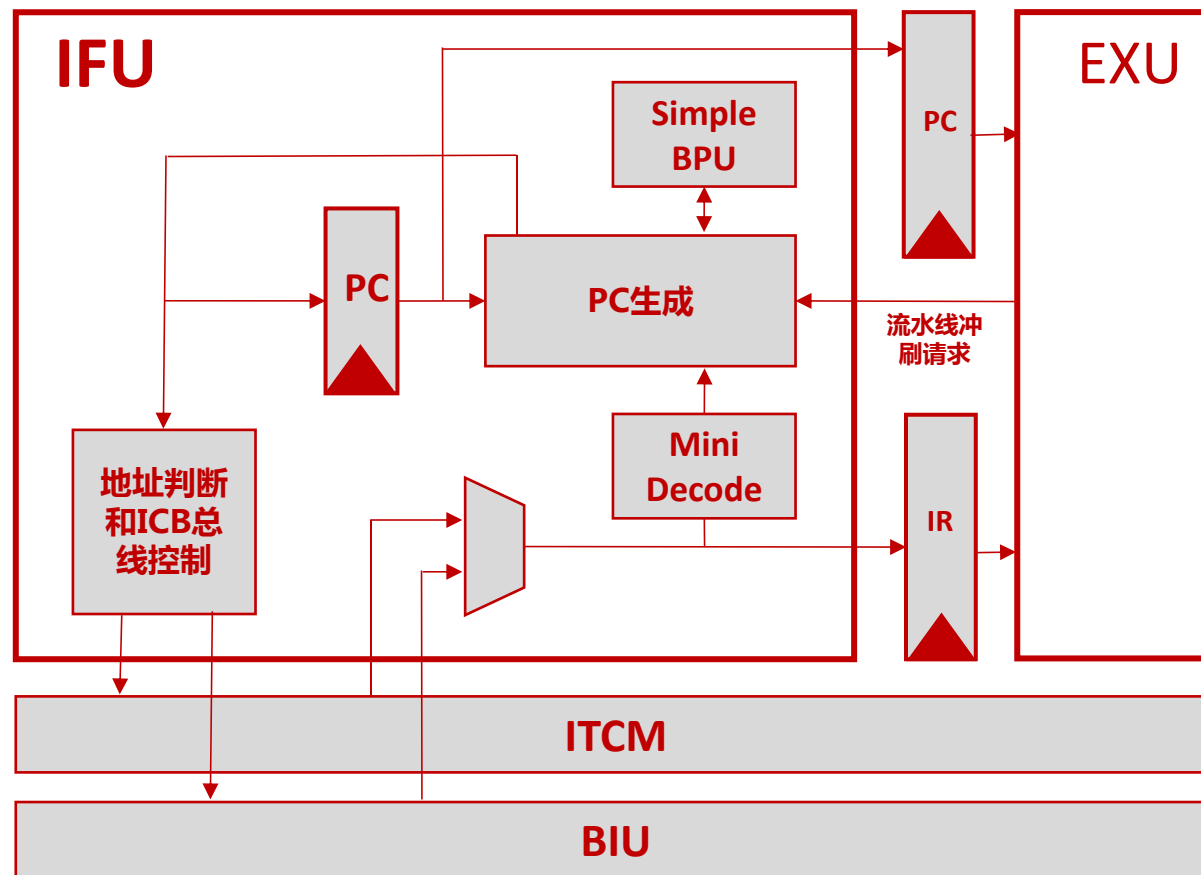
蜂鸟E203是个非严格意义的**二级流水线结构**，第一级为**IFU**完成取指，第二级则包含了**EXU**进行译码，执行和写回。但对于通过**LSU**进行访存，执行浮点运算等长指令(多周期指令)，以及通过**EAI**访问协处理器扩展则由第三级或更多级完成，因此严格讲蜂鸟E203是一个**变长流水线结构**。



蜂鸟E203取指单元

蜂鸟E203的设计强调超低功耗，小面积和实时性，因此取舍后，采用了如下设计：

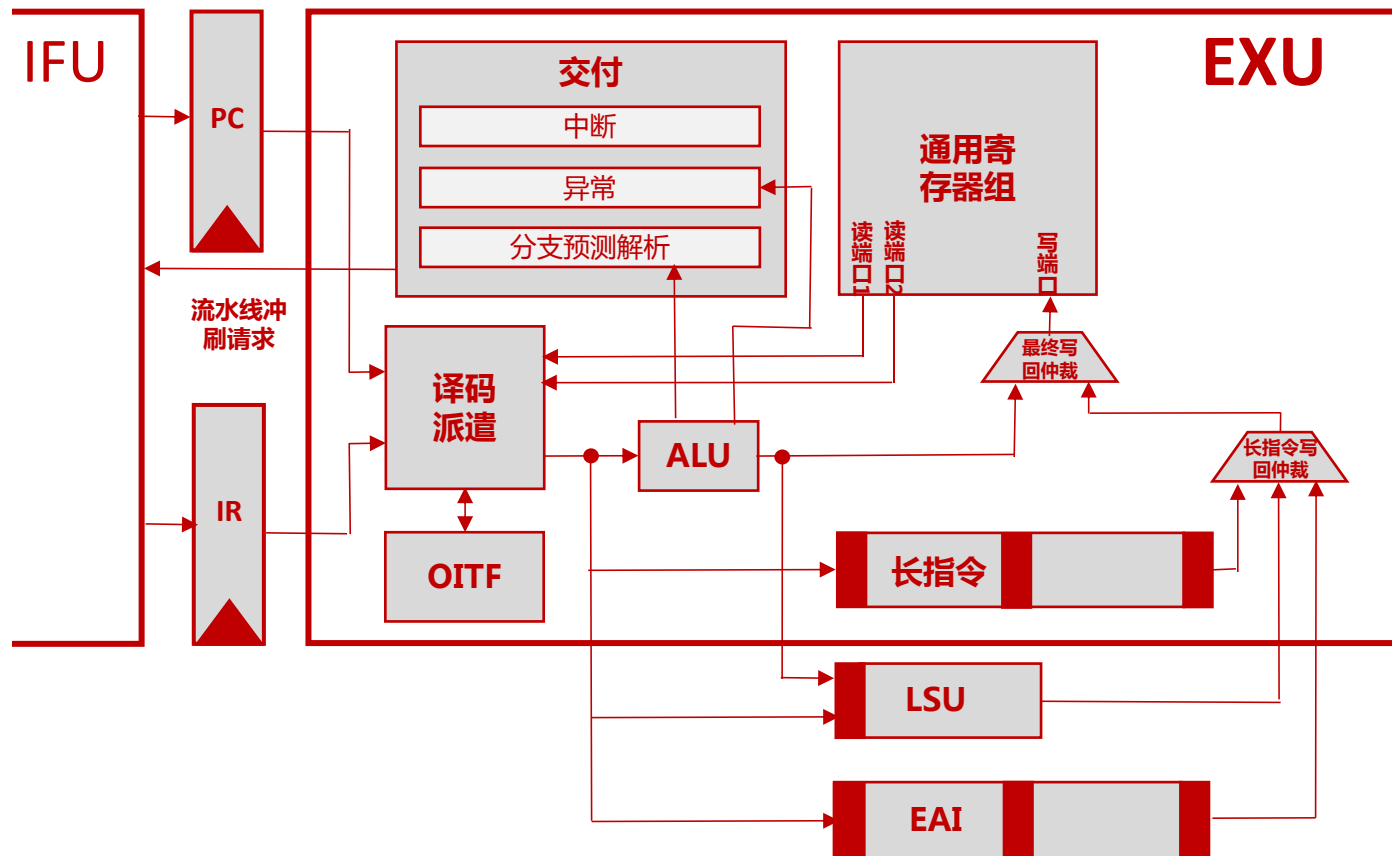
- 对取回的地址进行简单译码(Mini-Decode)以确定是普通指令还是分支跳转指令，以提供PC生成做决策；
- 当遇到分支跳指令，采用简单的**静态预测**，而舍弃动态分支预测保证在效率的情况下逻辑足够简单；
- 假设代码足够小，并且保证运行的实时性，选择了**ITCM**而舍弃了I-Cache；
- 假设大部分情况下无需访问外部存储器，因此针对**ITCM**优化，而舍弃针对BIU的优化。



蜂鸟E203执行单元

蜂鸟E203的执行单元设计思路如下：

- 将取指单元IFU通过IR寄存器发送给EXU的指令进行译码和派遣；
- 通过译码出的操作数寄存器索引(Index)读取通用寄存器组中的数据；
- 通过OITF维护指令的数据相关性；
- 将指令派遣(Dispatch)给ALU, 长指令，LSU和EAI等运算单元执行；
- 将指令交付；
- 将指令运算的结果写回通用通用寄存器组。

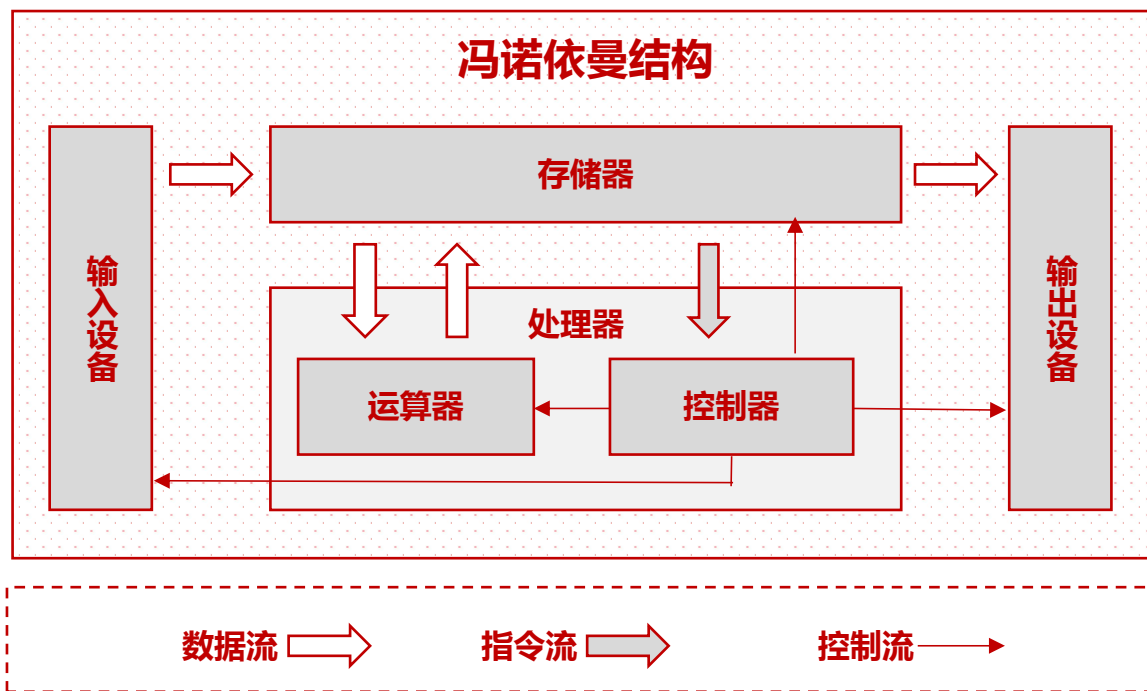


蜂鸟E203存储架构设计

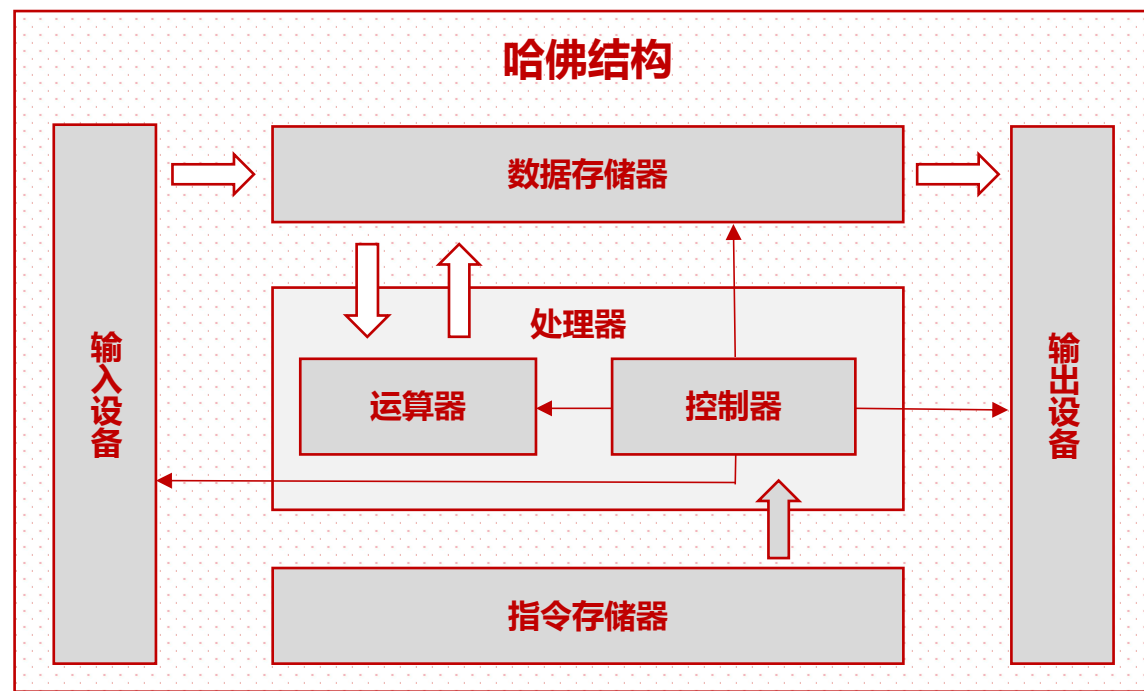
根据现代计算机体系结构是一定会有存储器来存放数据和指令的，存在**冯诺依曼结构(或称为普林斯顿)**和**哈佛结构**两者设计风格，前者把指令和数据放在同一个存储空间实现简单可有效减少处理器功耗和面积，后者将指令和数据分开来存放大大提高了数据吞吐率但结构比冯诺依曼结构复杂。蜂鸟可以根据配置成不同的结构

- 同时具备ITCM和DTCM的配置，并分别用来存储指令和数据，这种配置构成了**哈佛结构**，也是蜂鸟E203的典型结构，后续讨论都是以此配置为基础；
- 同时具备ITCM和DTCM的配置，利用ITCM除了存储指令外还可以存储数据，在这种配置下，实际为**冯诺依曼和哈佛混合结构**；
- 只保留ITCM和配置，由于ITCM除了存储指令外也可以用于存储数据，在这种配置下其实为**冯诺依曼结构**。

冯诺依曼结构



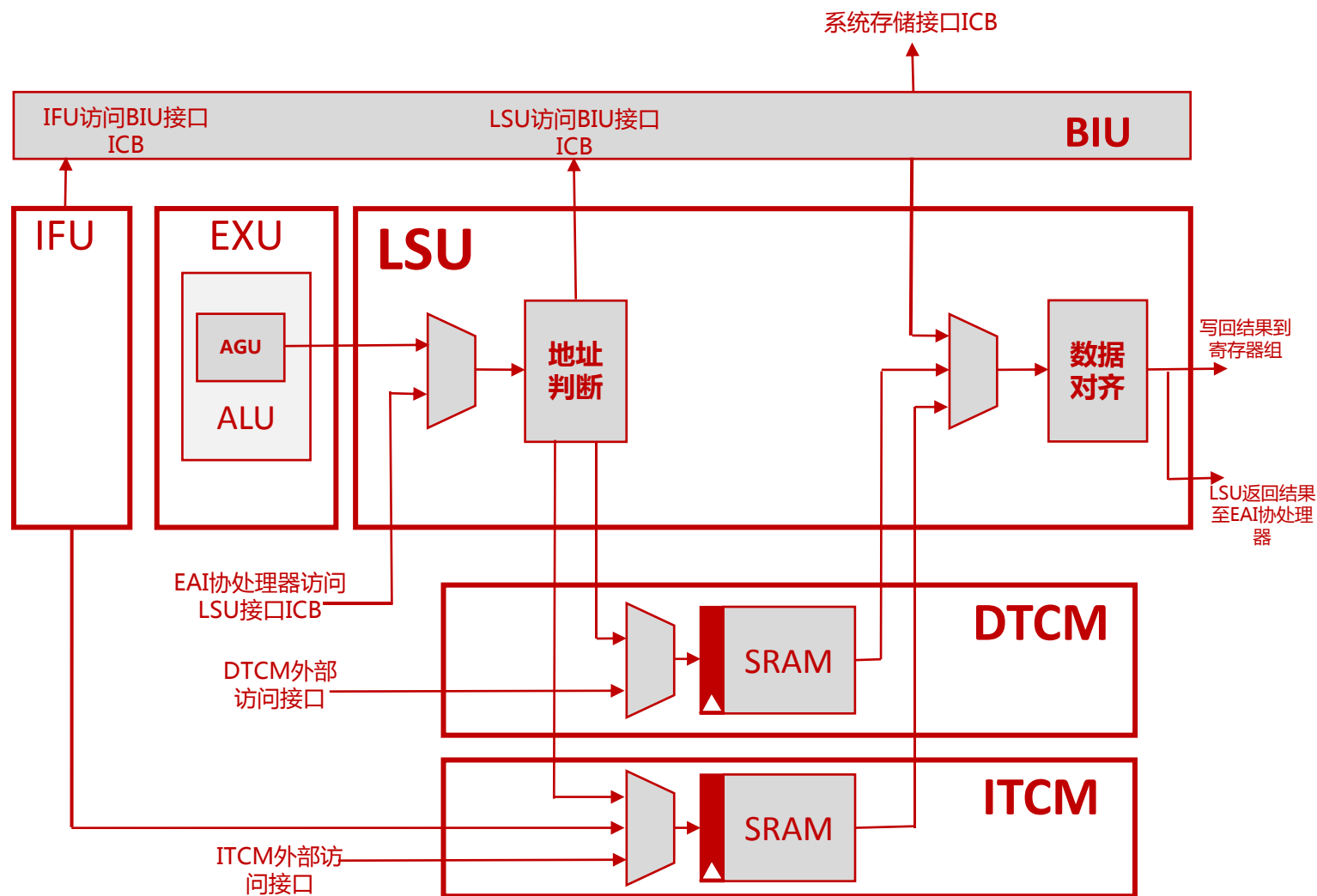
哈佛结构



蜂鸟E203存储架构设计

蜂鸟E203的存储子系统设计：

- 采用哈佛结构，ITCM作为指令存储部件，DTCM作为数据存储部件（如果使用ITCM存储数据的能力实为混合结构）；
- 因为假设面向的应用领域代码足够小，因此没有采用带缓存(Cache)的设计；
- LSU作为存储器访问的控制模块；
- ALU的子模块AGU为读(Load)和写(Store)指令以及A扩展生成存储器访问地址。读写指令需要将其第一个寄存器索引的源操作数和符号位扩展立即数相加得到最终地址，会用到加法器，为了节省面积复用ALU的加法器用于访存地址的计算；
- 可以通过BIU的系统存储接口ICB访问外部存储器。



蜂鸟E203总线协议ICB

常见的片上总线协议包括：

- **AXI** (Advanced eXtensible Interface)
 - 优：应用广泛，高吞吐率，易于添加流水线级数，支持多个滞外交易
 - 缺：控制复杂，硬件开销大，不适用超低功耗SoC
- **AHB** (Advanced High Performace Bus)
 - 优：应用广泛，高性能功耗低
 - 缺：不易添加流水线级数，不支持多个滞外交易，握手协议别扭，跨时钟域或者整数倍时钟域困难
- **APB** (Avanced Peripheral Performance Bus)
 - 优：控制简单，便于与低速周边外设连接
 - 缺：吞吐率低，不适合用作数据总线
- **TileLink**，由伯克利大学定义
 - 优：高性能，支持缓存一致性
 - 缺：目前应用不广泛，实现复杂，不适用超低功耗SoC



蜂鸟E203结合AXI总线和AHB总线的优点定义了一种自定义总线协议

ICB(Internal Chip Bus), 用于处理器内核内部以及SoC总线使用，其特点包括

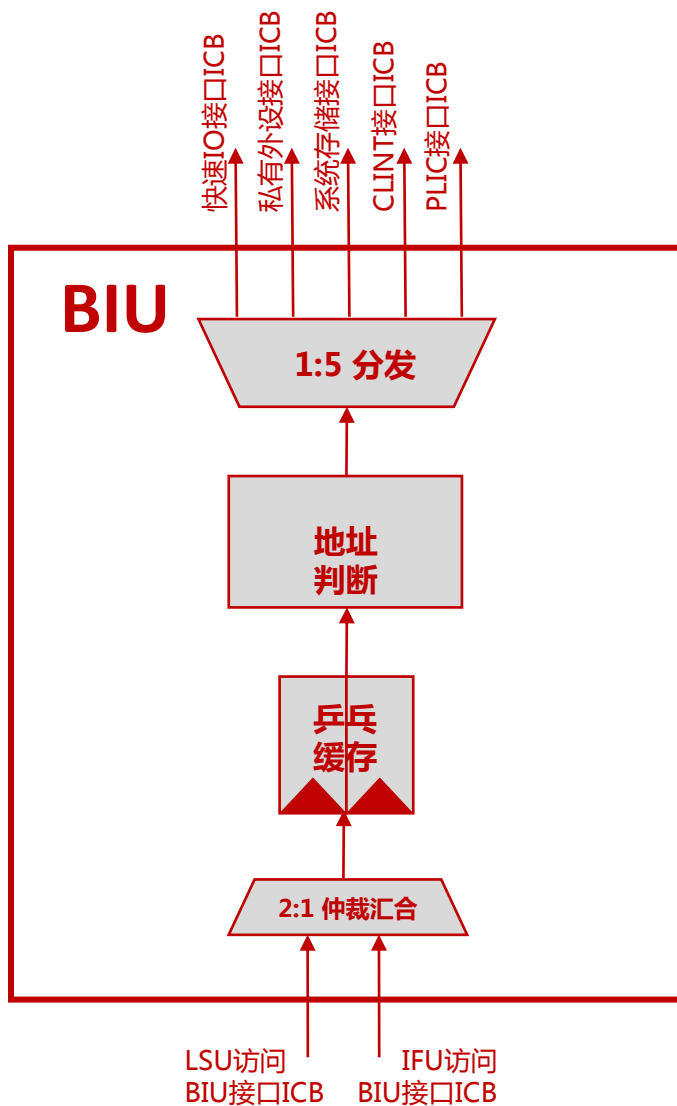
- 分离的地址和数据阶段(同AIX)；
- 采用地址区间寻址，支持任意的主从数目，如一主一从，一主多从，多主一从等拓扑结构(同AXI)；
- 每个读或写操作在地址通道上产生地址，而不是像AXI只产生起始地址(同AHB)；
- 支持地址非对齐的数据访问，使用字节掩码来控制部分写操作（同AXI）；
- 支持多个滞外交易(同AXI)；
- 不支持乱序返回乱序完成，反馈通道必须按顺序返回结果(同AHB)；
- 易于添加流水线级数以获得高频的时序；（同AXI）；
- 协议定义简单，易于桥接转换成其他常见片上总线。

	通道	方向	宽度	信号名	介绍
I C B 总 线 信 号	命令通道	输出	1	icb_cmd_valid	主设备向从设备发送读写请求信号
		输入	1	icb_cmd_ready	从设备向主设备返回读写接受信号
		输出	DW	icb_cmd_addr	读写地址
		输出	1	icb_cmd_read	读或是写操作的指示
		输出	DW	icb_cmd_wdata	写操作的数据
		输出	DW/8	icb_cmd_wmask	写操作的字节掩码
	反馈通道	输入	1	icb_rsp_valid	从设备向主设备发送读写反馈请求信号
		输出	1	icb_rsp_ready	主设备向从设备返回读写反馈接受信号
		输入	DW	icb_rsp_rdata	读反馈的数据
		输入	1	icb_rsp_err	读或者写反馈的错误标志

蜂鸟E203总线接口单元

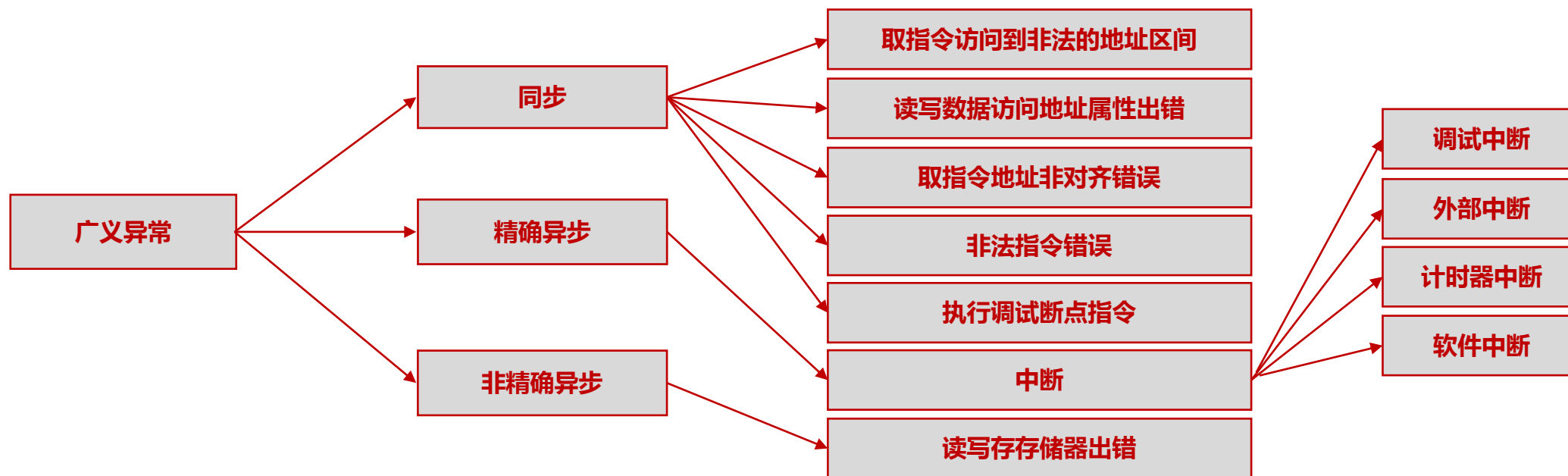
蜂鸟E203的总线接口单元BIU(Bus Interface Unit)设计如下

- 具备两组输入ICB总线接口，分别来自LSU和IFU，用于访问外部存储和设备；
- 两个输入ICB总线接口经过一个ICB汇合模块，将其汇合成一组ICB总线，采用的仲裁机制为优先级仲裁，LSU总线具有更高优先级；
- 为砍断外界与处理器核内部的时序路径，加入一组乒乓缓存(Ping-Pong Buffer)；
- ICB总线通过命令通道判断要访问的地址区间后使用ICB分发模块将命令分发给不同的外部接口；
- ICB汇合和ICB分发模块的FIFO深度默认为1，因此默认只支持一个滞外交易，该配置是为了减少面积开销；
- 为了防止IFU访问设备地址产生不可预知的结果，如果发生则通过起反馈通道返回错误标志结果。



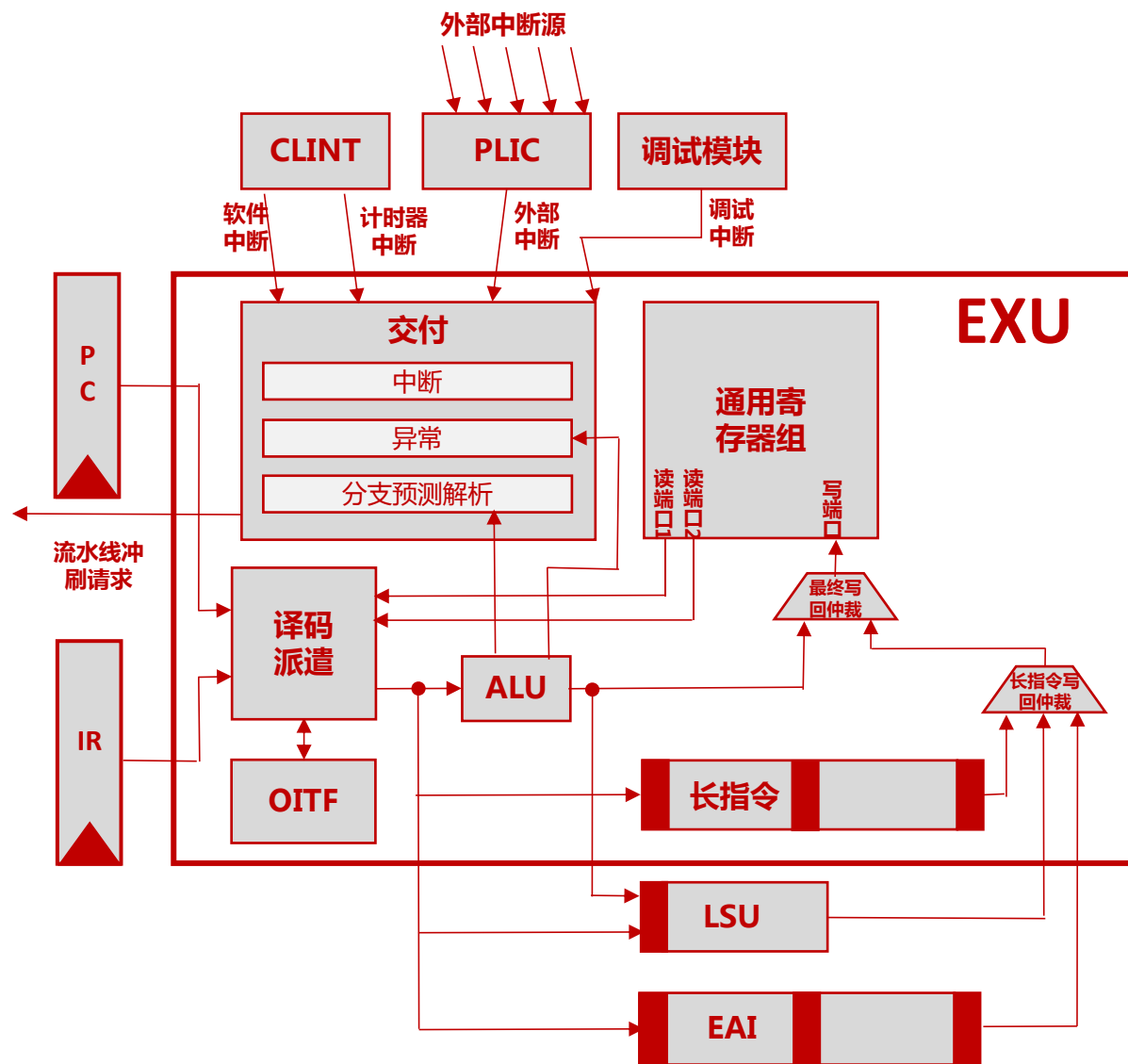
蜂鸟E203的中断和异常

中断机制和**异常**机制即在中断和异常发生时，处理器将暂停当前正在执行的程序，转而执行中断和异常处理程序，返回时处理器恢复之前被执行的程序。中断和异常最大的区别是前者是由外部因素导致，而后者则是由处理器内部事件或程序执行中的事件引起，例如硬件或程序故障。由于对于处理器而言中断和异常的处理机制基本上一致，所以合称**广义异常**。广义异常可以分为**同步异常**和**异步异常**，前者原因可以精确定位到某一条执行指令，后者每次发生时指令的PC可能不一样，例如中断便是一种异步异常。异步异常分为**精确异步异常**和**非精确异步异常**两类，前者在响应异常后的处理器状态能够精确反映某一条指令的边界，而后者则由于指令执行被打断而边界模糊。



蜂鸟E203的中断和异常

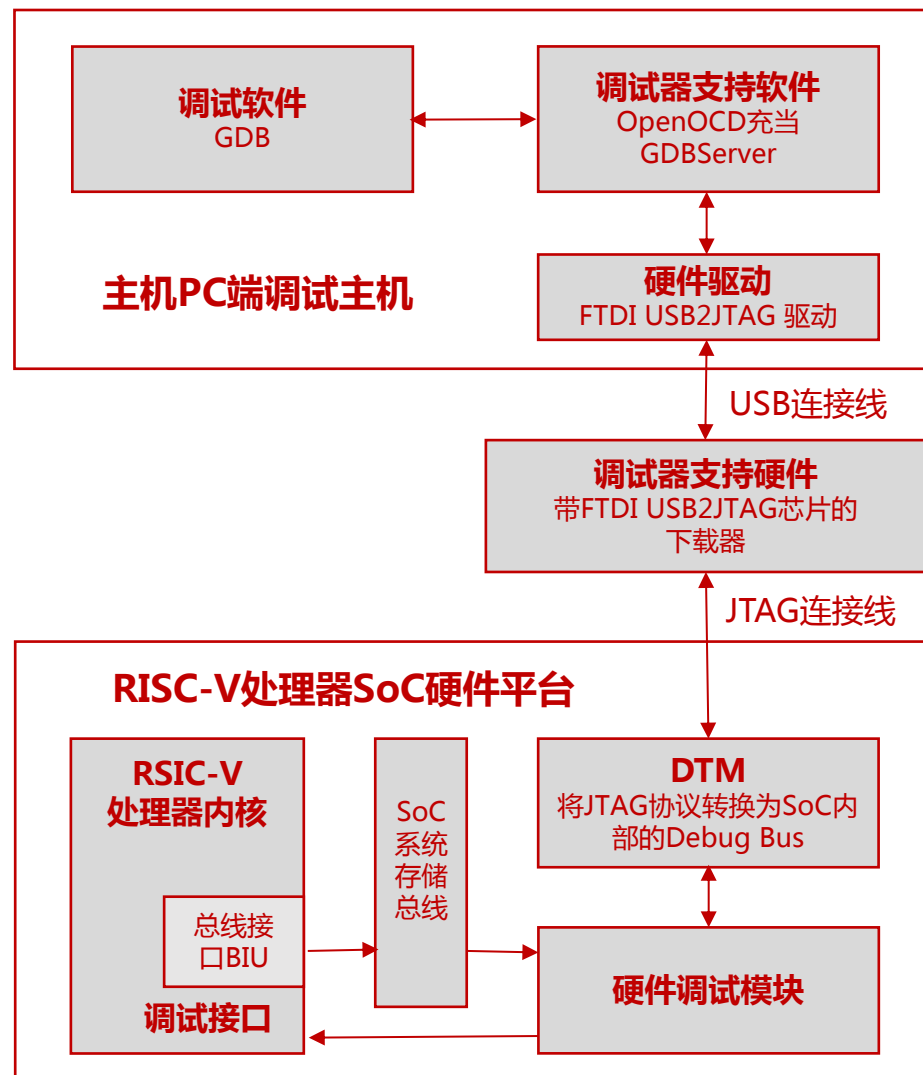
- 蜂鸟E203只支持RISC-V架构中的机器模式(Machine)，而不支持用户模式(User Mode)和监督模式(Supervisor Mode)，因此只支持模式相关的中断和异常类型；
- 蜂鸟E203只实现了RISC-V架构定义的软件中断，计时器中断和外部中断3种基本中断类型，并没有实现自定义中断；
- 蜂鸟E203的mtvec寄存器最低位的MODE域仅支持模式0，即所有异常响应时处理器均跳转到BASE域指示的PC地址；
- 关于如何进入异常，退出异常以及异常服务程序，等更多细节请参考《手把手教你设计CPU》第13章。



蜂鸟E203的调试机制

蜂鸟E203的调试机制的硬件实现严格依据0.11版本的调试架构文档(另有0.13版本)实现，其交换调试机制如下(未支持跟踪调试)：

- 开发人员在调试主机上通过GDB设置某程序的断点，GDB通过右图所示路径由JTAG接口访问处理器的硬件调试模块，对其下达命令希望在某PC设置一个断点；
- 硬件调试模块请求处理器内核停止，然后修改存储器中那个PC地址的指令，将其替换为设置断点指令即ebreak，再让处理器恢复执行；
- 当恢复执行后，在执行到那个PC对应的ebreak指令时，会导致异常进入调试模式的异常处理程序(一段在Debug-Rom中定义的固定程序) 调试模块发现这一状态改变后，将该信息显示出来。由于主机端的GDB软件一直在检测调试模块，得知处理器核已经停在了断点处时，便把对应的信息显示在GDB界面上。
- 当开发人员希望读取某个寄存器的值，则通过GDB对调试模块下达命令。调试模块立即开始对处理器内核进行控制，将寄存器值读出，并将此信息显示出来，主机端GDB在监测调试模块状态时得知此信息，便显示在其界面上；
- 当希望退出调试模式，则可以调用dret指令来触发。



蜂鸟E203的低功耗的诀窍

低功耗设计是一个会涉及到从软件到工艺的各个层面系统工程，针对每个层面采取不同的策略，蜂鸟E203的低功耗机制如下

- **系统层面**，配套SoC整体可以分为调试域，主域和常开域三个电源域，通过关闭不同的电源域进入不同的低功耗模式；
- **处理器层面**，通过实现wfi指令，来让处理器内核进入休眠，并通过中断唤醒；
- **单元层面**，内核几个主要功能单元均配置了独立的时钟门控，一旦单元出于空闲的周期，即自动地将时钟关闭，从而节省动态功耗；
- **寄存器层面**，通过寄存器编码为D触发器引出使能信号方便进行时钟门控，减少数据通路翻转和数据通路不复位来减少功耗；
- **锁存器层面**，可以通过配置将寄存器组模块配置为基于锁存器的实现，从而降低面积和功耗；
- **SRAM层面**，选择单口64位SRAM，64位位宽SRAM用于构建ITCM时可以优先减少SRAM读写减少动态功耗，SRAM均配备了独立的时钟门控单元在空闲时关闭减少动态功耗；
- **组合逻辑层面**，通过复用资源例如ALU中的数据通路等以减少面积，并尽量减少组合逻辑的反转率；
- **工艺层面**，始用特殊的工艺单元库，这里不做讨论。

软件层面低功耗

系统层面低功耗

处理器层面低功耗

单元层面低功耗

寄存器层面低功耗

锁存器层面低功耗

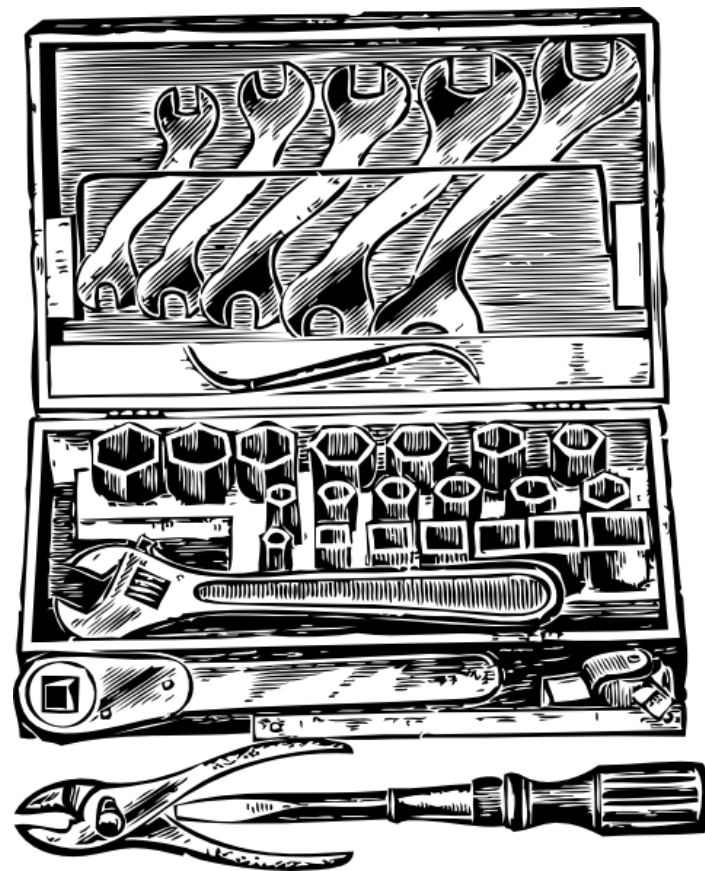
SRAM层面低功耗

组合逻辑层面低功耗

工艺层面低功耗

蜂鸟E203的扩展性

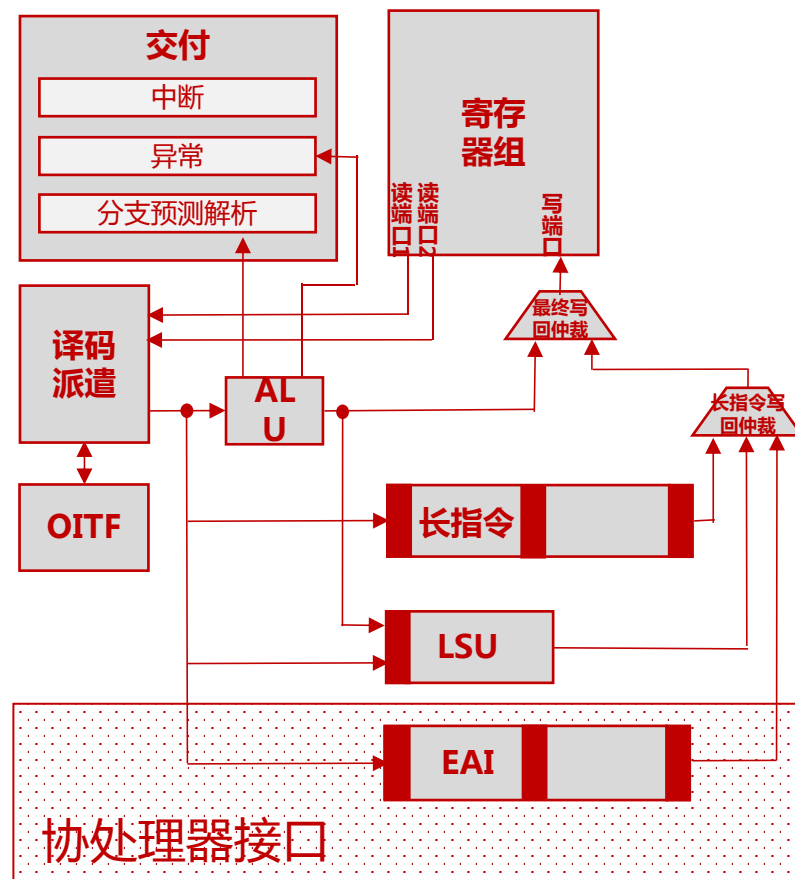
专用领域架构 (DSA, Domain Specific Architecture)逐渐成为趋势，其目标是提升计算的能效比。如何达成这个目标？其中一个核心思想是术业有专攻，在硬件领域便是**用专用硬件满足特定领域需求**，但和一般ASIC硬件化不同，DSA是要满足的是一个领域的需求解决一类问题而非单一问题，因此能够满足灵活和专用性的折衷。就处理器领域而言，DSA可以被解释成Domain Specific Accelerator, 即在通用处理的基础上，扩展出面向某些领域的加速器，以提升解决该领域问题的效率。



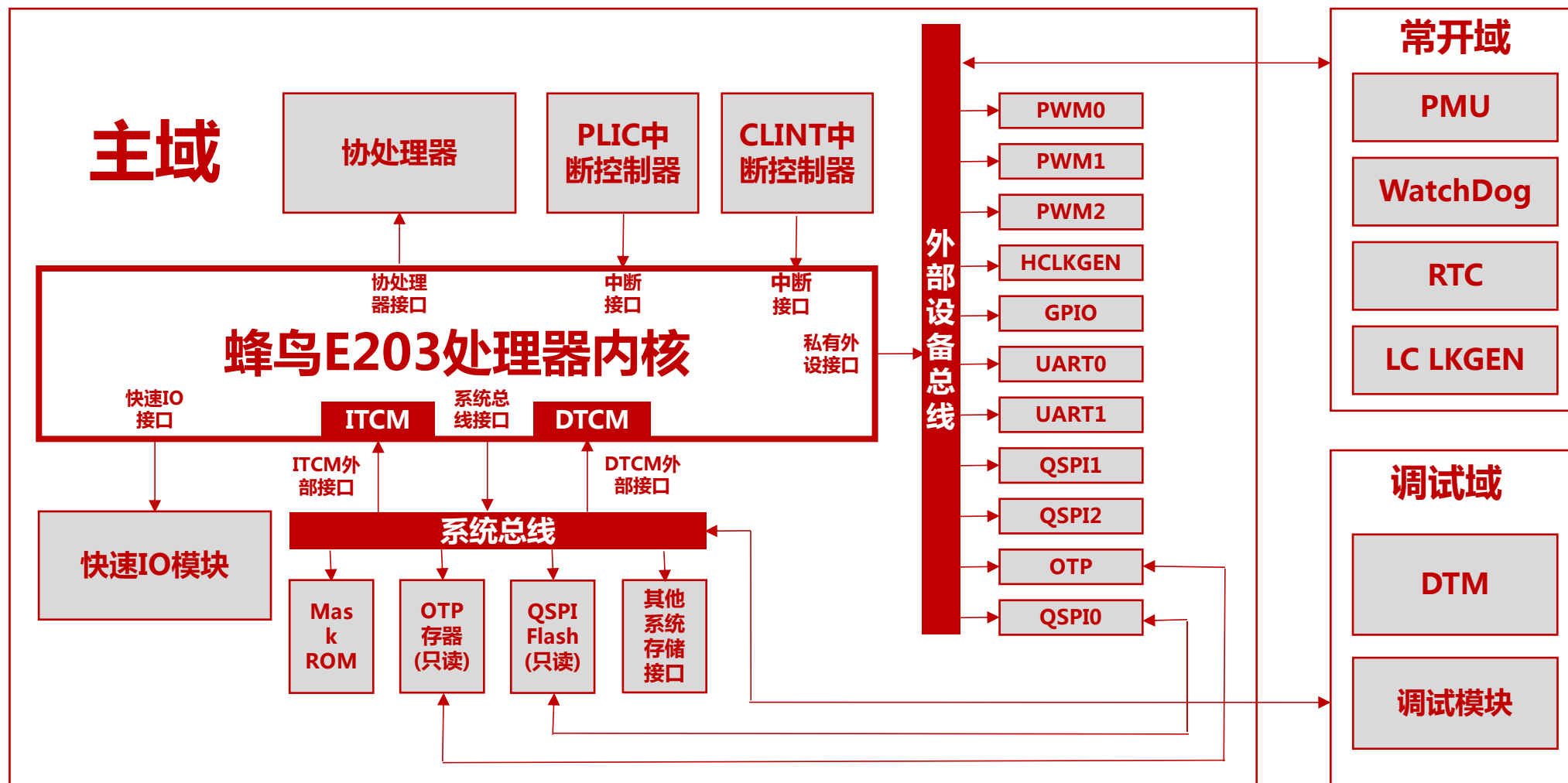
蜂鸟E203的扩展性

RISC-V架构的一个显著特征便是开放的可扩展性便于实现DSA，具体体现在预留了指令编码空间和预定义的Custom指令。蜂鸟E203的协处理器机制EAI(Extension Accelerator Interface)即扩展加速器接口借鉴了开源RISC-V处理器Rocket Core的协处理器接口RoCC(Rocket Custom Coprocessor)，要点如下：

- EAI指令编码，使用RV32的4个Custom指令组，可以自定义出共计512条带两个源寄存器和一个目标寄存器的指令，当少于2个源寄存器或无需目标寄存器时可以自定义更多指令
- EAI接口信号，包含请求，反馈，存储器请求，存储器反馈，存储器占用5个通道共计22个信号；
- EAI流水线接口，主处理器判断是否是Custom指令并决定是否需要从源寄存器读取参数，以及需要回写到目标寄存器，并维护数据依赖的正确性，然后在EXU通过EAI接口派发该指令到协处理器，协处理器对指令进一步译码进行处理并把执行结果通过EAI接口返回，主处理器收到反馈后将此指令从流水线中退役并将结果协会寄存器组；
- EAI存储接口，在LSU中为EAI预留了专用访问接口，协处理器可以访问主处理器能够寻址的数据存储器资源；
- EAI时序接口以及协处理器参考示例请参考《手把手教你设计CPU》16.3和16.4节。



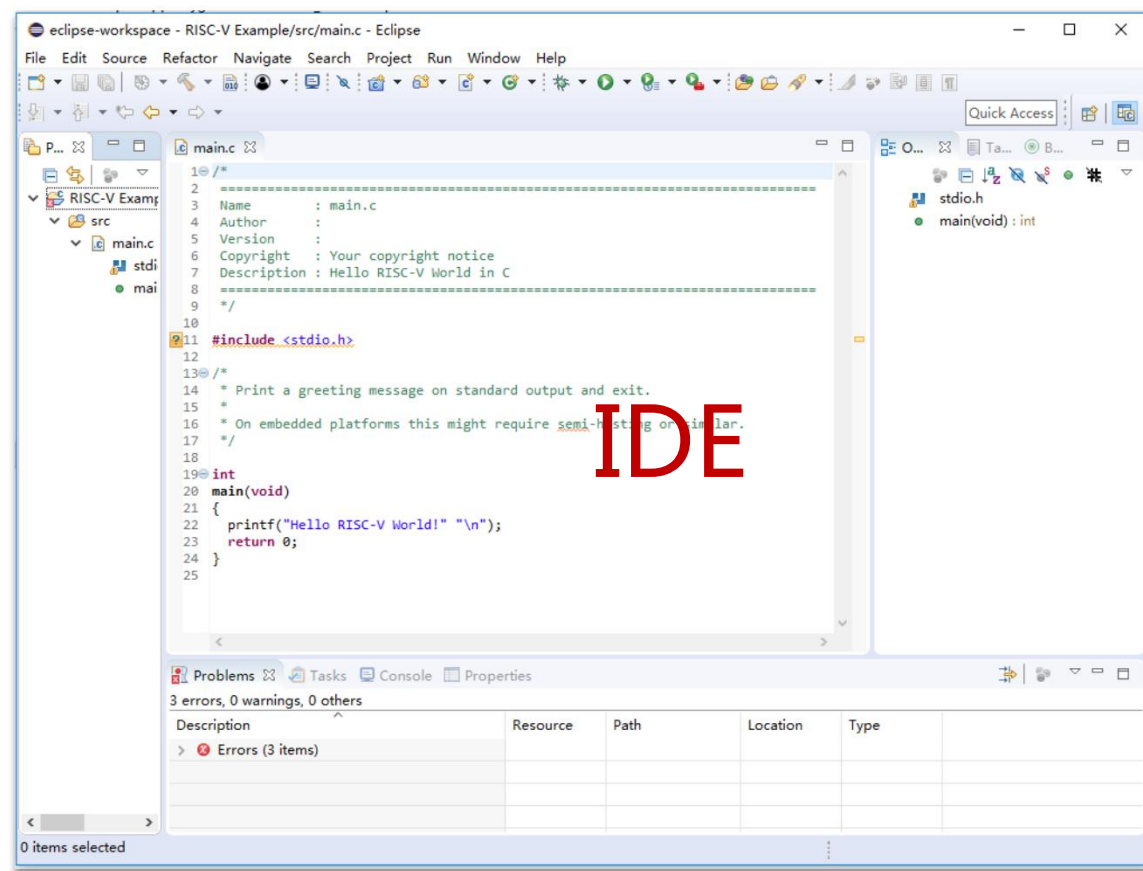
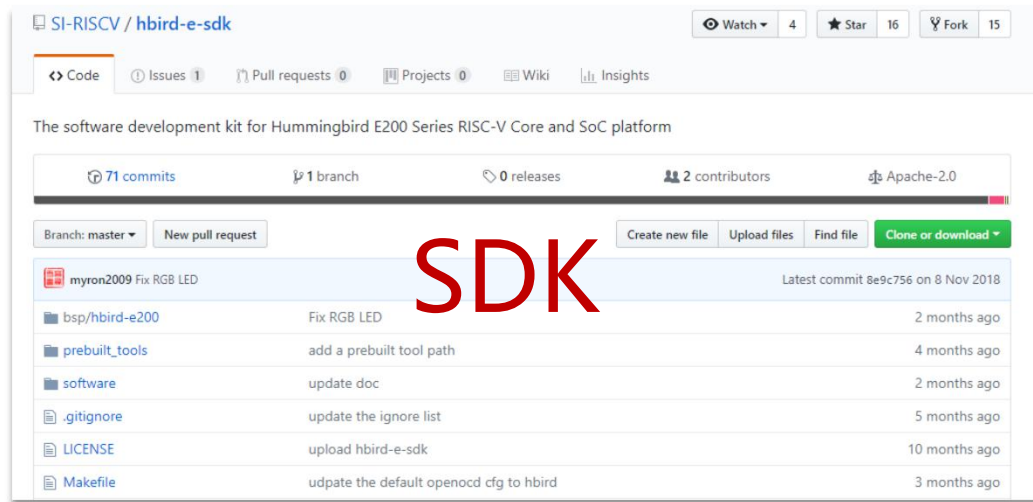
蜂鸟E203 SoC



蜂鸟E203 开发环境

蜂鸟E203开发环境由RISC-V工具链，板级支持包和图形化的IDE构成。

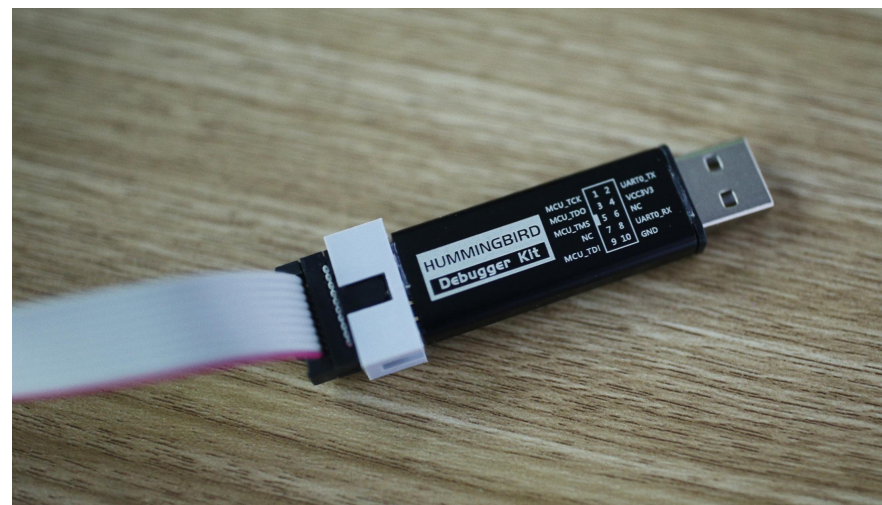
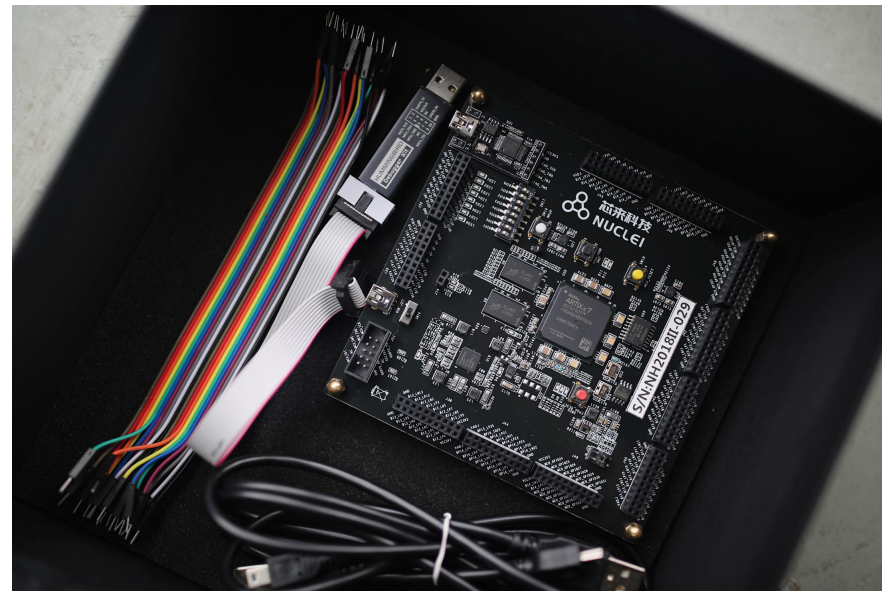
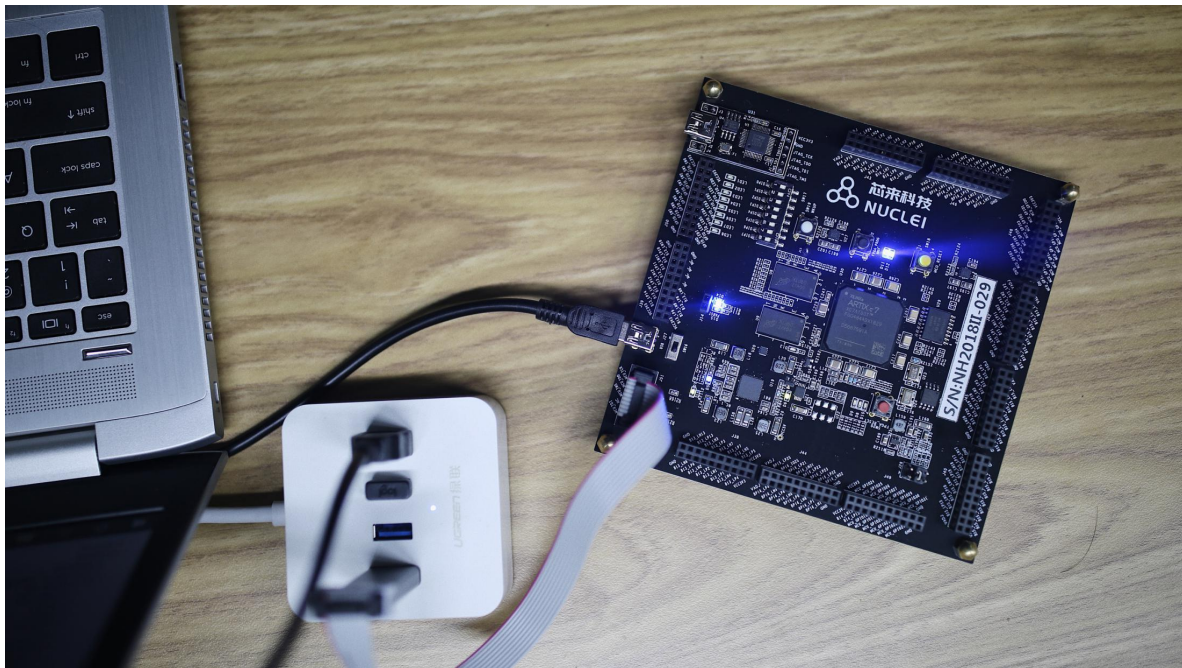
- 为了让用户能够非常容易地使用RISC-V处理器开发软件，蜂鸟E203不仅开源了SoC平台，也开源了配套的软件开发平台——HBird-E-SDK平台，其主要由板级支持包和若干软件示例构成。
- 蜂鸟E203可以使用开源的MCU Eclipse IDE进行基于Windows的图形化开发。



专用FPGA开发板和调试器

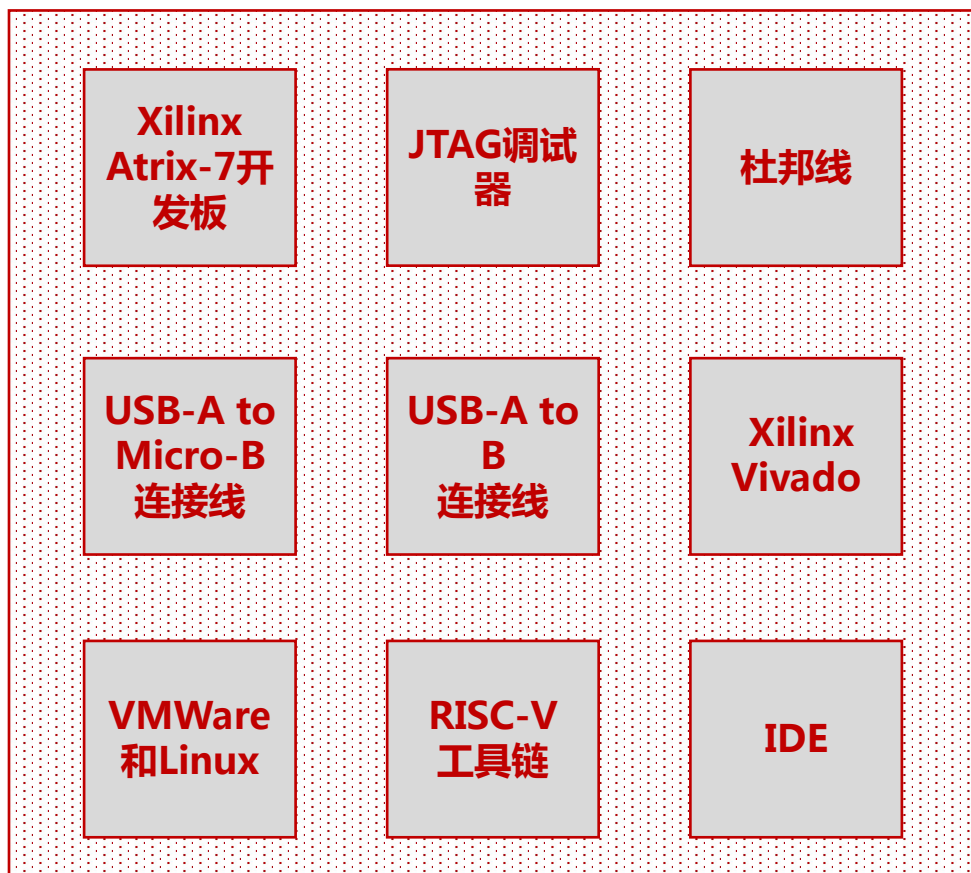
为了便于初学者能够快速地学习RISC-V CPU设计和RISC-V嵌入式开发，蜂鸟E203开源项目为MCU原型SoC定制了：

- 基于FPGA的专用学习开发板
- 专用JTAG调试器

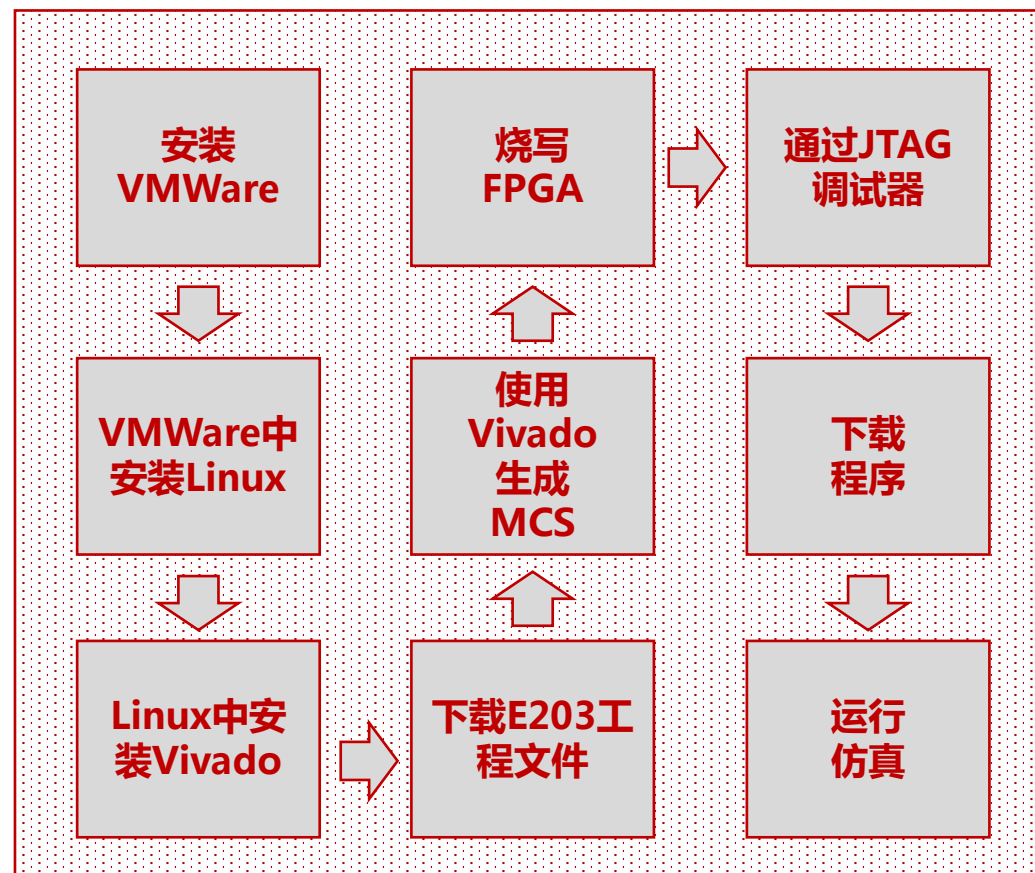


通过FPGA进行仿真

环境配置



执行步骤



蜂鸟E203的贡献

●推动了RISC-V在国内的普及

由于中文化翔实而系统资料，且平民化的Verilog开发，让本土爱好者能够快速的上手和入门RISC-V。

●彻底打通了软硬件的底层边界

蜂鸟E203不仅开源内核，还配套开源SoC、开源软件SDK、开源IDE、调试器、开源RTOS移植等，形成一个从最底层硬件到上层软件之间的完整链条，是绝佳的科教素材。

●展示了工业级CPU IP的设计风格

展示了工业级稳健和成熟的设计方法、思路和设计代码，并深入进行剖析。

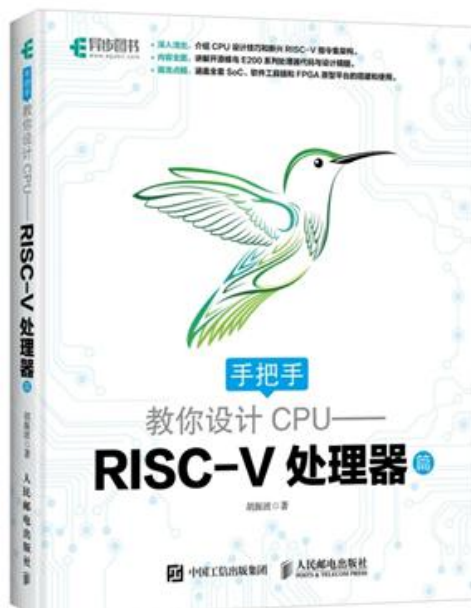


Agenda

- 1 关于ISA和RISC-V
- 2 开源RISC-V内核蜂鸟E203
- 3 RISC-V的中文书籍**
- 4 蜂鸟E203软硬件DEMO视频
- 5 RISC-V的几个误区
- 6 RISC-V的未来趋势
- 7 RISC-V的商业本质
- 8 芯来科技介绍

RISC-V的中文设计书籍

《手把手教你设计CPU——RISC-V处理器篇》



第一部分 CPU 与 RISC-V 综述

- 第 1 章 一文读懂 CPU 之三生三世..... 2
- 第 2 章 大道至简——RISC-V 架构之魂..... 29
- 第 3 章 乱花渐欲迷人眼——盘点 RISC-V 商业版本与开源版本..... 46
- 第 4 章 开源 RISC-V——蜂鸟 E200 系列超低功耗 Core & SoC..... 54

第三部分 使用 Verilog 进行仿真和在 FPGA SoC 原型上运行软件

附录部分 RISC-V 架构详述

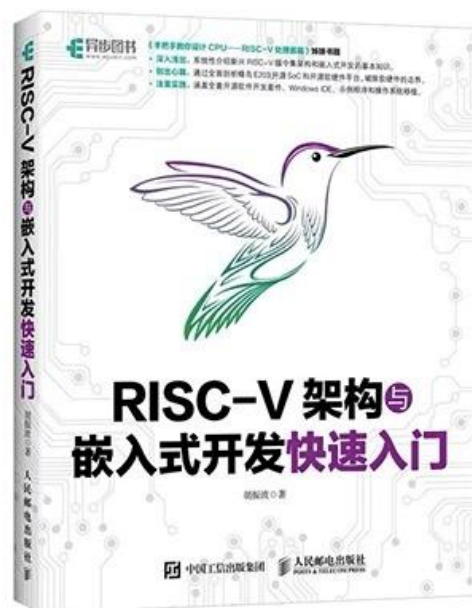
- 附录 A RISC-V 架构指令集介绍..... 342
- 附录 B RISC-V 架构 CSR 寄存器介绍..... 374
- 附录 C RISC-V 架构的 PLIC 介绍..... 384
- 附录 D 存储器模型背景介绍..... 392
- 附录 E 存储器原子操作指令背景介绍..... 397
- 附录 F RISC-V 指令编码列表..... 400
- 附录 G RISC-V 伪指令列表..... 404

第二部分 手把手教你使用 Verilog 设计 CPU

- 第 5 章 先见森林,后观树木——蜂鸟 E200 设计总览和顶层介绍..... 65
- 第 6 章 流水线不是流水账——蜂鸟 E200 流水线介绍..... 78
- 第 7 章 万事开头难吗——一切从取指令开始..... 88
- 第 8 章 一鼓作气,执行力是关键——执行..... 117
- 第 9 章 善始者实繁,克终者盖寡——交付..... 161
- 第 10 章 让子弹飞一会儿——写回..... 170
- 第 11 章 哈弗还是比亚迪——存储器架构..... 178
- 第 12 章 黑盒子的窗口——总线接口单元 BIU..... 203
- 第 13 章 不得不说的故事——中断和异常..... 217
- 第 14 章 最不起眼的,其实是最难的——调试机制..... 246
- 第 15 章 动如脱兔,静若处子——低功耗的诀窍..... 260
- 第 16 章 工欲善其事,必先利其器——RISC-V 可扩展协处理器..... 276

RISC-V的中文嵌入式书籍

《RISC-V架构与嵌入式开发快速入门》



第 1 章	进入 32 位时代，谁能成为下一个 8051	1
第 2 章	开源蜂鸟 E203 超低功耗 RISC-V Core 与 SoC	25
第 3 章	大道至简——RISC-V 架构之魂	29
第 4 章	RISC-V 中断和异常	43
第 5 章	开源蜂鸟 E203 MCU SoC 总体介绍	59
第 6 章	开源蜂鸟 E203 MCU SoC 外设介绍	78
第 7 章	开源蜂鸟 E203 MCU 开发板与调试器	143
第 8 章	编译过程简介	145
第 9 章	嵌入式开发特点与 RISC-V GCC 工具链	158
第 10 章	RISC-V 汇编语言程序设计	171
第 11 章	基于 HBird-E-SDK 平台的软件开发与运行	188
第 12 章	开源蜂鸟 E203 MCU 的更多示例程序	226
第 13 章	Windows IDE 集成开发调试环境	245
第 14 章	开源蜂鸟 E203 MCU 开发板移植 RTOS	283
附录 A	RISC-V 架构指令集介绍	287
附录 B	RISC-V 架构 CSR 寄存器介绍	321
附录 C	RISC-V 架构的 PLIC 介绍	331
附录 D	存储器模型背景介绍	340
附录 E	存储器原子操作指令背景介绍	345
附录 F	RISC-V 指令编码列表	349
附录 G	RISC-V 伪指令列表	353

Agenda

- 1 关于ISA和RISC-V
- 2 开源RISC-V内核蜂鸟E203
- 3 RISC-V的中文书籍
- 4 蜂鸟E203软硬件DEMO视频**
- 5 RISC-V的几个误区
- 6 RISC-V的未来趋势
- 7 RISC-V的商业本质
- 8 芯来科技介绍

DEMO

蜂鸟RISC-V开源处理器软硬件演示视频地址

<https://v.qq.com/x/page/m08324sj90v.html>

Agenda

- 1 关于ISA和RISC-V
- 2 开源RISC-V内核蜂鸟E203
- 3 RISC-V的中文书籍
- 4 蜂鸟E203软硬件DEMO视频
- 5 RISC-V的几个误区**
- 6 RISC-V的未来趋势
- 7 RISC-V的商业本质
- 8 芯来科技介绍

误区1:此开放非彼开放

- 以企业为主导的处理器开放组织并不是第一次，譬如：**OpenSPARC**，**OpenPower**，**OpenMIPS**
- RISC-V是第一次以非盈利中立组织来维护指令集标准，且向全世界进行开放
- 正是因为RISC-V这种更加纯粹的开放，RISC-V以前所未有的速度被普及和接纳。以蜂鸟E203为例，其启动和传播完全出于个体的自发行为，如果是用其他商用架构，那是不可能做到的



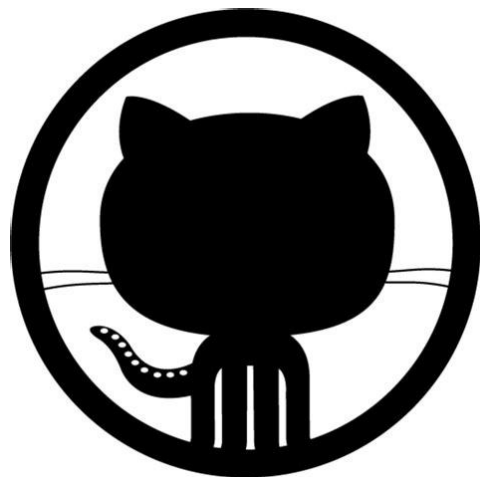
误区2:此FREE非彼FREE

- Free的英文存在着多重含义，可以是**自由(Freedom)**也可以是**免费**
- RISC-V的Free是**自由而非免费**，这种是驱动创新的原动力
- 世上无免费的午餐，大家都爱免费，但夸大宣传RISC-V的免费对国内产业发展无裨益



误区3:此OpenSource非彼OpenSource

- Linux的OpenSource开源的是源码
- RISC-V的OpenSource开源的是文字标准
- RISC-V和Linux没有类比性，RISC-V没有主线源码库
- RISC-V跟DDR，USB等等硬件标准更加有类比性。DDR等标准不断升级，RISC-V的ISA标准在不断的衍进。由于处理器内核是最复杂且软硬件综合一体的IP，所以相比普通的接口类标准，有其特殊性



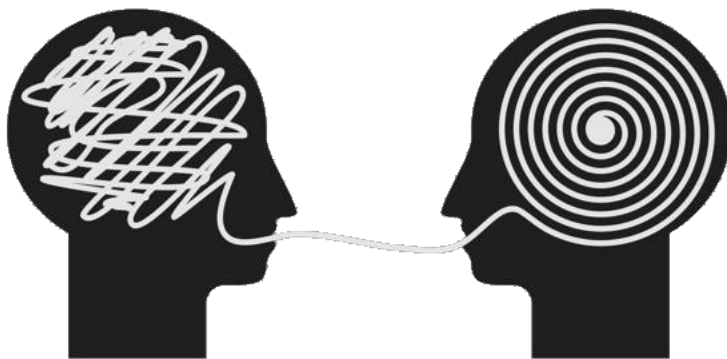
误区4:RISC-V只适用于IoT

- 背景知识：
 - 嵌入式领域内的绝对生态壁垒——二进制兼容——并不存在
 - 嵌入式设备的软硬件一体性和源代码重编译特性决定了其只存在生态相对壁垒
- IoT的生态相对门槛更低，所以RISC-V的接纳难度更低，拥有广泛的增量市场和定制化需求，因此是RISC-V介入的机会点
- RISC-V标准和技术本身并不局限于IoT领域



误区5:RISC-V等于新的设计语言和方法学

- 背景：
 - 新的设计语言能够加速编写代码的速度
 - 新的方法学能够带来新的想象空间
 - 芯片产业更趋于稳健
- RISC-V只是一份开放指令集标准，与新的设计语言和方法学无关
- 本土产业更需专注解决核心产业有和无的问题，将RISC-V与新的设计语言和设计方法打包宣传对于RISC-V的推广并无裨益



误区6:RISC-V等于不要版税

- 背景：
 - 人类已知的商业模式：**卖产品，卖服务，流量变现**
 - 芯片IP行业依赖的版税模式，本质上等于卖服务
- RISC-V只是一种开放标准，与具体的商业模式无关



RISC-V唯一的标签

kāi fàng
开 放

Agenda

- 1 关于ISA和RISC-V
- 2 开源RISC-V内核蜂鸟E203
- 3 RISC-V的中文书籍
- 4 蜂鸟E203软硬件DEMO视频
- 5 RISC-V的几个误区
- 6 RISC-V的未来趋势**
- 7 RISC-V的商业本质
- 8 芯来科技介绍

趋势论:处理器领域的几次变革和趋势

- 趋势论

- 愈发**开放**的趋势
- 愈发**平民化**的趋势

- 历史进程：

- **处理器诞生之初**：以IBM / Sun为代表的整机和服务，开启了人类信息化的大潮；
- **第一次变革**：以Intel / AMD为代表的x86处理器芯片，极大地推进了PC和服务器的普及和繁荣；
- **第二次变革**：以ARM / MIPS为代表的商用处理器架构授权+IP，奠定了嵌入式和移动手持设备井喷的基础；
- **第三次变革**：以RISC-V为代表的开放标准。

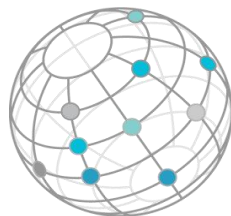


RISC-V顺势而生

- 开放的趋势：RISC-V的目标是成为一种由基金会维护的开放通用的指令集架构
 - 整机 → 处理器芯片 → 处理器IP → RISC-V标准
 - 因其开放，得到了诸多大型公司和社区的支持，因此生态发展迅速
 - 当生态建设不再由一家公司而是由全行业推动时，其速度远远高于以往由单一商业公司推动，且成本被均摊
- 平民化的趋势：技术上利用后发优势，所以简洁，模块化和可扩展
 - 满足AIoT万亿级市场的差异化需求



基金会维护



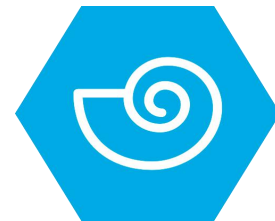
架构开放



社区活跃



后发优势



简洁设计



模块化可扩展

Agenda

- 1 关于ISA和RISC-V
- 2 开源RISC-V内核蜂鸟E203
- 3 RISC-V的中文书籍
- 4 蜂鸟E203软硬件DEMO视频
- 5 RISC-V的几个误区
- 6 RISC-V的未来趋势
- 7 RISC-V的商业本质**
- 8 芯来科技介绍

RISC-V的商业本质

背景观点一：处理器生态中最昂贵的成本是什么

- 教育成本：人民普遍的熟悉程度
- 接受成本：人民愿意投入的时间

昂贵到无法用金钱造就和购买



RISC-V的商业本质

背景观点二：处理器生态的核心是什么？

- 对长远来看，生态的核心不是存在多少已有的Codes和方案
- 长远来看，生态的核心是“人心”和“人性”
 - “人心”对自由的向往会导致其更倾向于开放的技术
 - “人性”的懒惰决定了处理器生态会存在主链效应



RISC-V的商业本质

背景观点三：没有门槛的舞台将造就是壁垒最高的舞台



绝对Open的舞台 → 催生出更加绝对的垄断

RISC-V的商业本质

开放将生态中最昂贵的成本降到几乎为0

- 开放——解决了生态中的核心问题

RISC-V离不开专业的商业公司

- 商业公司的专注性，推进产业的落地
- 开源世界促进生态，并反哺商业公司

开放标准对门槛的双重影响

- 降低了生态的参与门槛
- 抬升了商业公司的门槛
- 催生出更加绝对的垄断

RISC-V商业公司的壁垒

- 绝对的技术和服务壁垒
- 其他的商业要素壁垒



RISC-V的商业本质

中国处理器IP产业最后一次机会

- 处理器产业变革的最后一次趋势
- 开放性大幅降低业余门槛但大幅抬高商业门槛
- 一旦新的商业格局重新形成，中国将机会甚微



RISC-V的商业本质

当前阶段RISC-V商用IP必备的五大要素

●专业、本地、服务

- 专业、优质、和及时的本地化服务，为客户的成功长期保驾护航

●充足的验证质量

- 一流的设计，成熟和稳健的设计方法，良好的代码可读性，以及充分的验证以保证质量。为客户的芯片集成与ASIC流程带来便利与信心
- 为客户提升产品研发效率，抢占市场先机，是对商用处理器IP的基本要求

●出色的PPA

- 出色的PPA (Power Performance Area) 参数和良好的可扩展性，为客户的产品带来出色竞争力和差异性

●代码密度足够高

- 优异的代码密度 (Code Density) ，为客户的芯片节省宝贵的片上SRAM或Flash存储空间从而降低其芯片成本

●完善的IDE

- 一流的图形化集成开发环境 (IDE , Integrated Development Environment) ，通过完善的功能以及优秀的体验提升用户的生产效率，这对于面向AIoT领域的芯片产品极为重要

Agenda

- 1 关于ISA和RISC-V
- 2 开源RISC-V内核蜂鸟E203
- 3 RISC-V的中文书籍
- 4 蜂鸟E203软硬件DEMO视频
- 5 RISC-V的几个误区
- 6 RISC-V的未来趋势
- 7 RISC-V的商业本质
- 8 芯来科技介绍**

芯来科技

- 芯来科技（Nuclei System Technology）创立于2018年，是中国大陆最有代表性的新创RISC-V处理器内核IP和解决方案公司。
- 公司从创立之初便聚焦RISC-V处理器内核研发，助力本土RISC-V生态建设，为本土产业赋能。
- 公司自成立以来发展迅猛，在短短数月之内便完成了团队搭建、千万级天使轮融资、新产品研发交付等关键步骤。
- 芯来科技目前是RISC-V基金会银级会员，中国RISC-V产业联盟副理事长单位，以及中国开放指令集生态（RISC-V）联盟会员单位。



想更深入地了解RISC-V?

关注我们



或

加入我们

芯来科技诚招热爱RISC-V的你加入，与这个充满激情团队一起快速成长。

这里有行业内有竞争力的薪资，有着未来可期的股权激励机制。这里大牛云集、高配团队、弹性工作、环境舒适、五险一金、佳节福利。

如果你想学到RISC-V的软硬件各方面知识与技能，如果你想在最短的时间内成长为一名技术大牛，这里有你想要的所有。

What are you waiting for ?

有意向的同仁，请将简历发送至hr@nucleisys.com

邮箱标题注明：姓名+应聘岗位。

招聘链接 请 扫



